

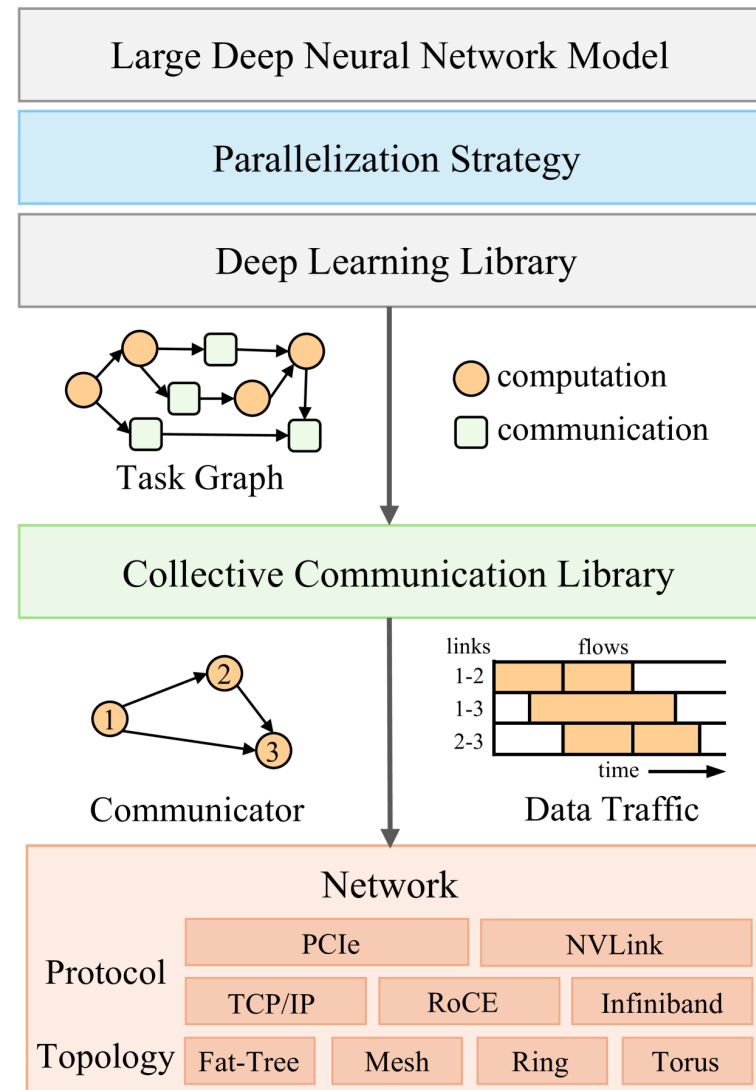
Introduction to High-Performance Machine Learning

Lecture 6

03/23/2026

Dr. Kaoutar El Maghraoui

Distributed Deep Learning



MPI and Distributed DL Communications Primitives

ML Performance Factors



System view:
Bottom-Up
(this course)

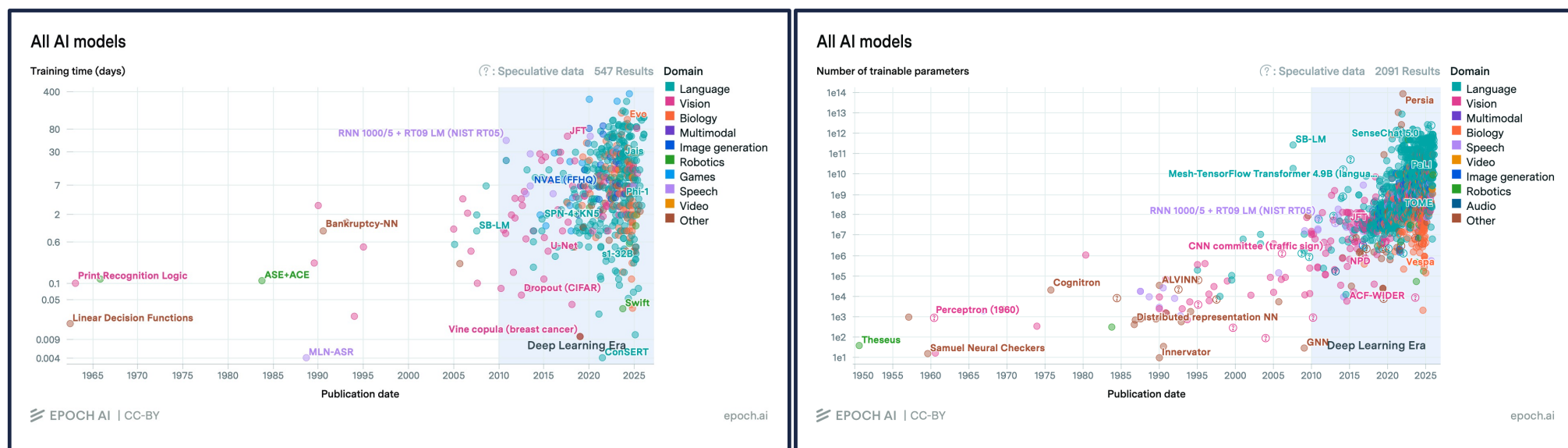


Algorithmic view:
Top-down



Modern Large Language AI Models are growing

1e14 = 100 Trillion Parameters



AI hardware parallelism is essential to meeting client performance and efficiency expectations.

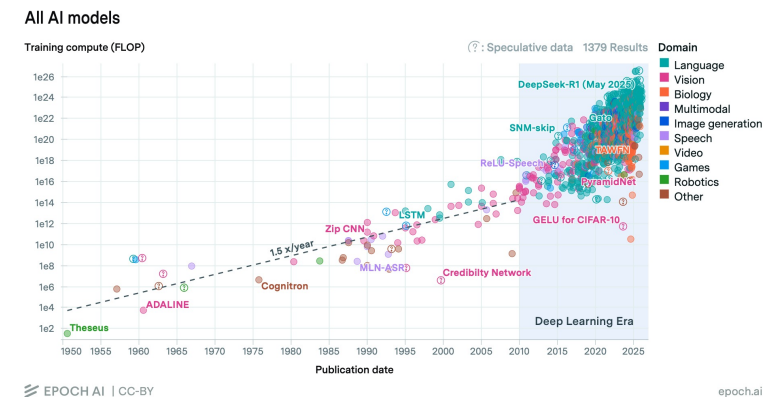
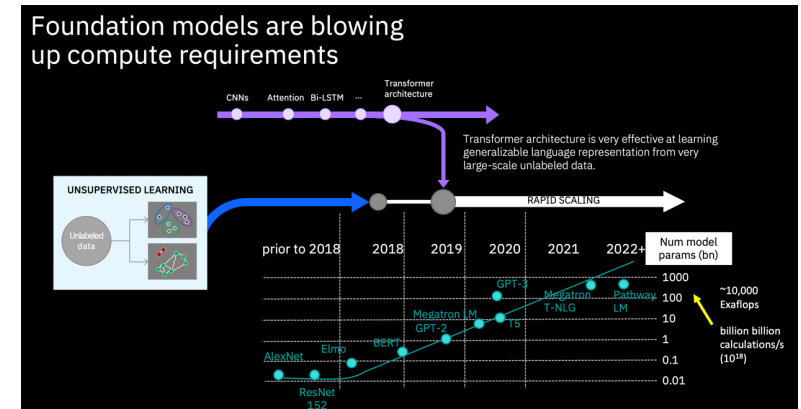
Performance: Returning an accurate response within an acceptable timeframe.

Efficiency: Total cost of ownership measured by inference throughput per watt.

Epoch AI, 'Data on AI Models'. Published online at epoch.ai.
Retrieved from 'https://epoch.ai/data/ai-models' [online resource]. Accessed 23 March 2026.

Why Distributed Deep Learning

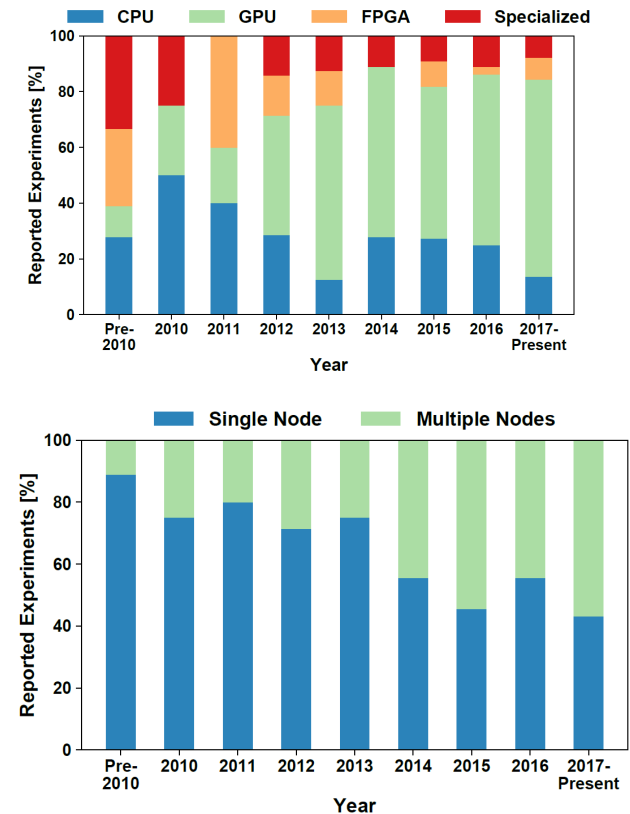
- Models are getting larger (GPT-4, LLaMA 3 405B, Gemini, Claude 3+)
- LLaMA 3 405B at fp16 = 810 GB — single H100 has only 80 GB
- Single GPU cannot handle memory, or compute demands of frontier models
- Need to scale across multiple GPUs and nodes
- Efficient communication is critical to maintain performance at scale
- Current scale: GPT-4/Gemini trained on 10,000–100,000+ GPUs
- Speed example: TSM video model, 50 hrs → 14 min with 1536 GPUs (211× speedup)



Epoch AI, 'Data on AI Models'. Published online at epoch.ai.
Retrieved from 'https://epoch.ai/data/ai-models' [online resource]. Accessed 23 March 2026.

Trend: From Parallel to Distributed DL

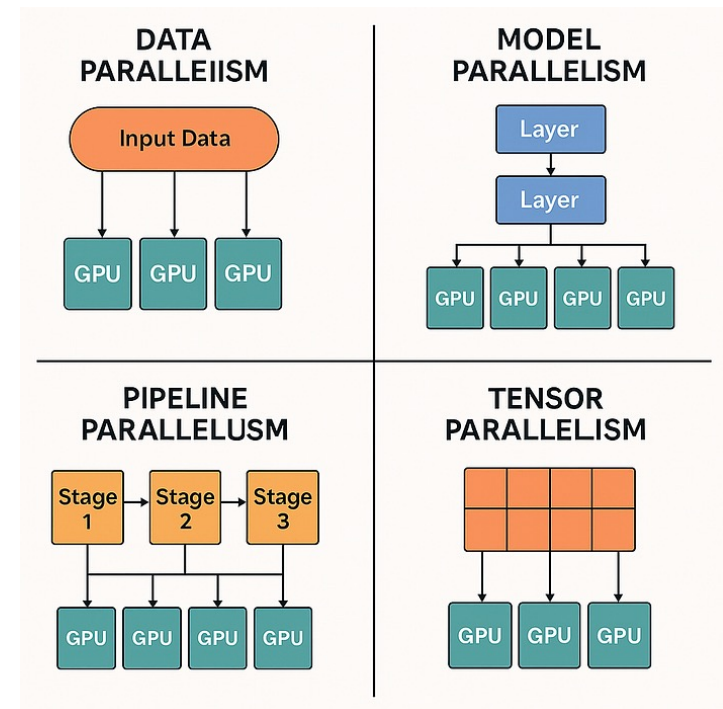
- Deep Learning has always exploited parallel architectures
- During the years:
 - DL has been moving towards distributed architectures
 - GPUs have become the default compute architecture
 - Models grew from millions → billions → trillions of parameters
- Node count and GPU cluster sizes have been growing since 2015
- MPI adoption has grown in AI as HPC and AI workloads converge
- **Modern LLM training requires 1K–100K GPU clusters running for weeks**



From: <https://arxiv.org/abs/1802.09941>

Key Concepts

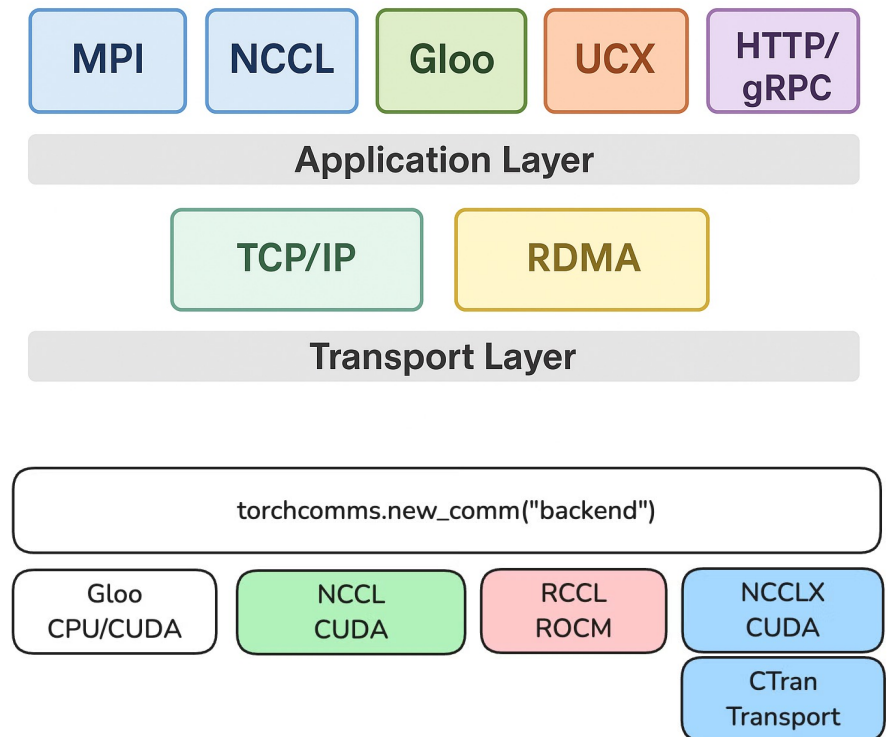
- **Data Parallelism:**
 - Split batches across GPUs; each holds a full model copy; gradients averaged via AllReduce
- **Model Parallelism:**
 - Split model layers/tensors across devices — required when model exceeds GPU memory
- **Pipeline Parallelism:**
 - Run different layers on different GPUs in a pipeline; microbatches flow through concurrently
- **Tensor Parallelism (NEW):**
 - Split weight matrices within a layer across GPUs — column-parallel and row-parallel linear layers
- **Collective Communication:**
 - AllReduce, Broadcast, Reduce-Scatter, AllGather — core synchronization primitives



<https://medium.com/@samanch70/beyond-data-parallelism-a-beginner-friendly-tour-of-model-pipeline-and-tensor-multi-gpu-a9fdf2e8176d>

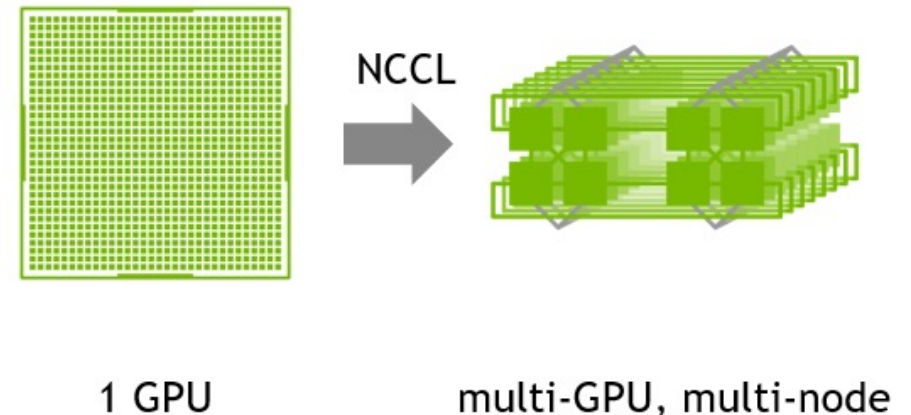
Communication Protocols Overview

- Used for inter-GPU / inter-node communication
- Impact latency, bandwidth, and scaling efficiency
- **High-level Libraries:**
 - NCCL — GPU-accelerated collectives, topology-aware, most popular
 - MPI — general-purpose, used in HPC and academic clusters
 - Gloo (Meta) — CPU-based fallback in PyTorch
- **Low-level Transports:**
 - UCX — RDMA/InfiniBand transport layer under MPI/NCCL
 - RDMA — kernel-bypass, ultra-low latency GPU-to-GPU transfers
 - SHARP — in-network reduction offloaded to InfiniBand switch hardware
 - TCP/IP — fallback for development and small-scale deployments



NCCL (NVIDIA Collective Communication Library)

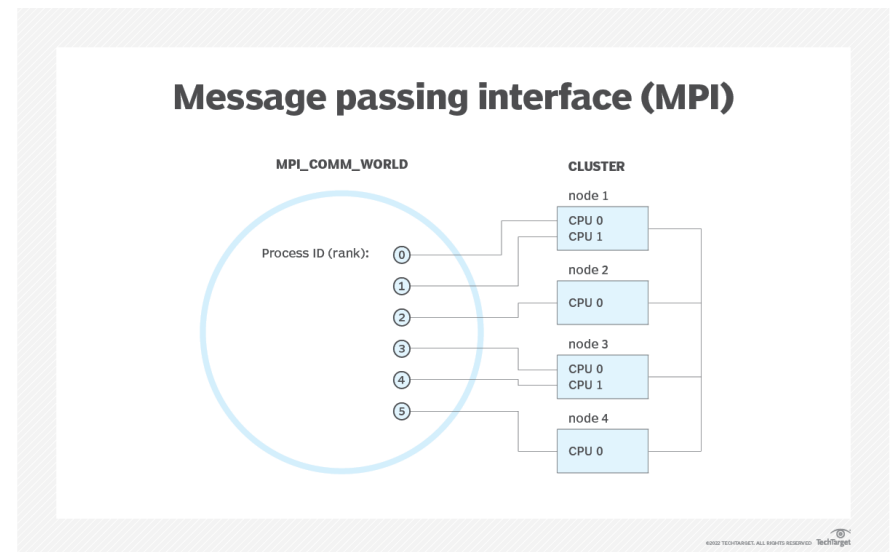
- GPU-accelerated collectives (AllReduce, Broadcast, Reduce-Scatter, AllGather)
- Topology-aware: detects NVLink, PCIe, InfiniBand; selects optimal path
- Backend for PyTorch DDP, FSDP2, DeepSpeed, Megatron-LM
- **Most popular communication library for NVIDIA-based training setups**
- **Key capabilities:**
 - NVSwitch fabric: 900 GB/s bidirectional bandwidth within a DGX H100 node
 - GPUDirect RDMA: GPU-to-GPU transfers across nodes without CPU involvement
 - Hierarchical AllReduce: NVLink intra-node, InfiniBand inter-node
 - SHARP integration: offload reduction ops to Quantum-2 in-network compute



<https://developer.nvidia.com/nccl>

MPI: Message Passing

- General-purpose HPC protocol for large-scale parallel systems
- Supports point-to-point and collective operations
- CUDA-aware MPI allows direct GPU buffer transfers without CPU staging
- Used in Horovod, academic HPC clusters, and HPC+AI convergence stacks
- **Version history:**
 - MPI 3.1 (June 2015) — non-blocking collectives
 - MPI 4.0 (2021) — Sessions API, partitioned communication
 - MPI 4.1 (November 2023) — persistent collectives, new buffer functions
 - MPI 5.0 (June 2025) — latest approved standard



<https://www.techtarget.com/searchenterprisedesktop/definition/message-passing-interface-MPI>

Gloo

- Developed by Meta
- CPU-based collectives (optional GPU support)
- Used in PyTorch for CPU-only or debugging
- **Fallback when NCCL unavailable**
- Supports AllReduce, Broadcast, Scatter, Gather
- Automatically selected as backend in PyTorch when NCCL is not available

pytorch/gloo

Collective communications library with various primitives for multi-machine training.

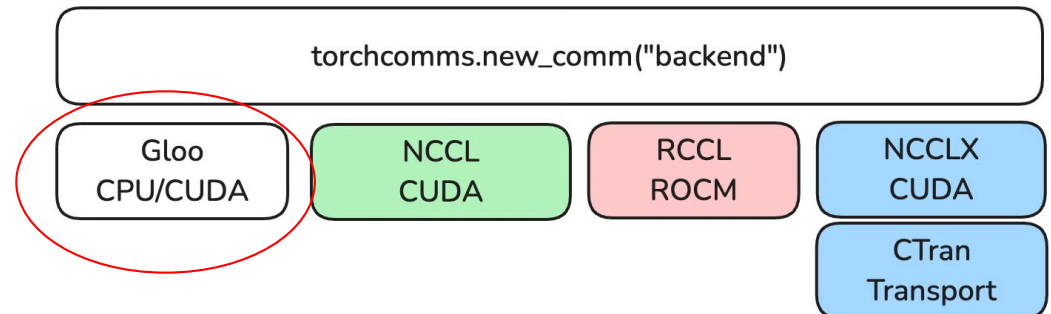


83
Contributors

65
Issues

1k
Stars

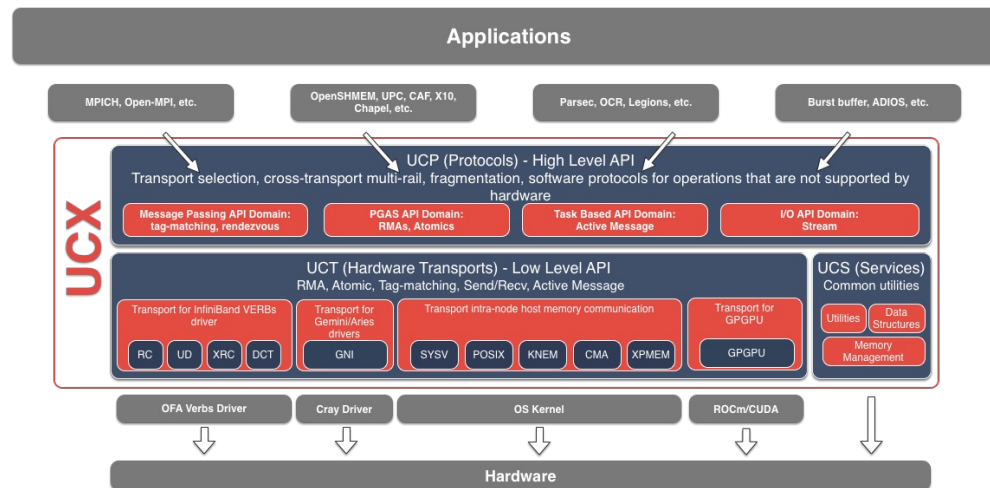
352
Forks



<https://pytorch.org/blog/torchcomms/>

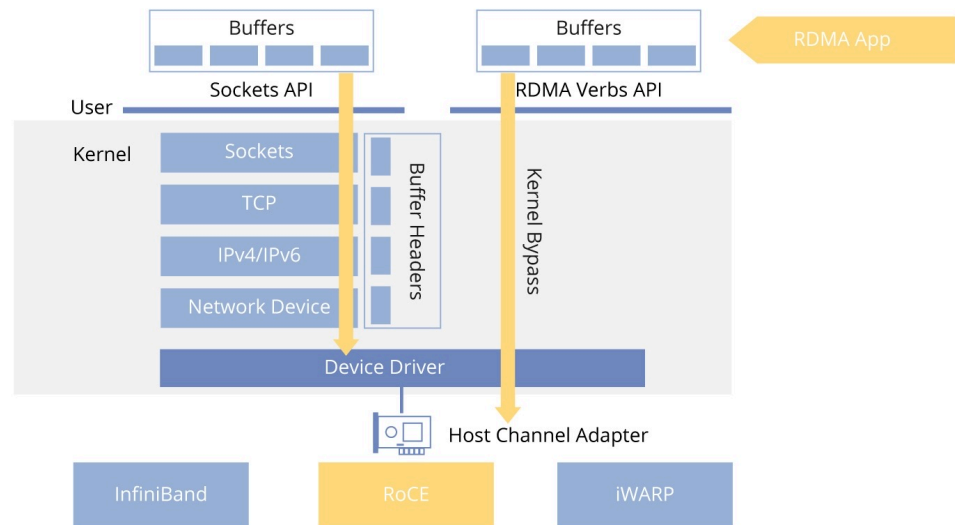
UCX: Unified Communication X

- Low-level transport layer (RDMA, InfiniBand, CUDA)
- Used under the hood in MPI/NCCL in HPC settings
- Supports zero-copy GPU-to-GPU transfers with ultra-low latency
- Popular in DGX clusters and DeepSpeed HPC deployments



RDMA (Remote Direct Memory Access)

- Enables direct memory transfers across nodes
- **Kernel-bypass, ultra-low latency — avoids OS overhead completely**
- Requires specialized hardware (e.g., Mellanox/NVIDIA ConnectX NICs)
- Underlying technology for UCX, MPI, and NCCL (via GPUDirect RDMA)
- InfiniBand HDR/NDR delivers 200–400 Gb/s with sub-microsecond latency
- GPUDirect RDMA allows GPU-to-GPU transfers without staging through CPU memory



<https://community.fs.com/blog/remote-direct-memory-access-rdma.html>

TCP/IP

- Ubiquitous networking protocol
- High latency, limited bandwidth compared to RDMA
- Used as fallback when InfiniBand/RDMA not available
- Suitable for development environments or small-scale cloud training

Protocols Used in Practice

Protocol	Popularity	Used In
NCCL	★★★★★	PyTorch DDP/FSDP2, DeepSpeed, Megatron
MPI	★★★★	Horovod, academic/HPC, HPC+AI convergence
Gloo	★★★	PyTorch CPU/debug fallback
UCX	★★★★	HPC, RDMA transport, DeepSpeed
SHARP	★★★★	In-network AllReduce on IB Quantum-2 switches
TCP/IP	★	Small-scale, cloud VMs, fallback

Communication in Distributed Deep Learning Frameworks

Design choices for communication:

- Established paradigms like MPI
- Specific libraries: NCCL, Gloo, UCX, SHARP
- Still need external communication for parameters (RDMA, InfiniBand)

Focus on Scale-up and Scale-out:

- Scale-up: intra-node via NVLink/NVSwitch (up to 900 GB/s)
- Scale-out: inter-node via InfiniBand (up to 400 Gb/s)

Modern framework landscape

- PyTorch (dominant) · DeepSpeed · Megatron-LM · JAX/XLA
- Caffe2, CNTK, MXNet — discontinued

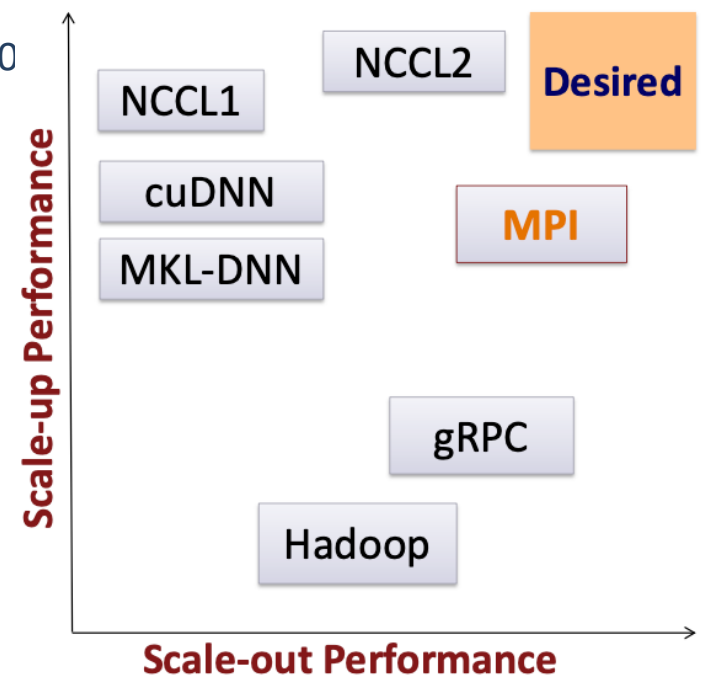
Scale-up and Scale-out

- **Scale-up: Intra-node communication**

- NVIDIA cuDNN, cuBLAS, NCCL within a node
- NVLink/NVSwitch: 900 GB/s bidirectional — DGX H100/H200 nodes
- CUDA cooperative groups and unified memory

- **Scale-out: Inter-node Communication**

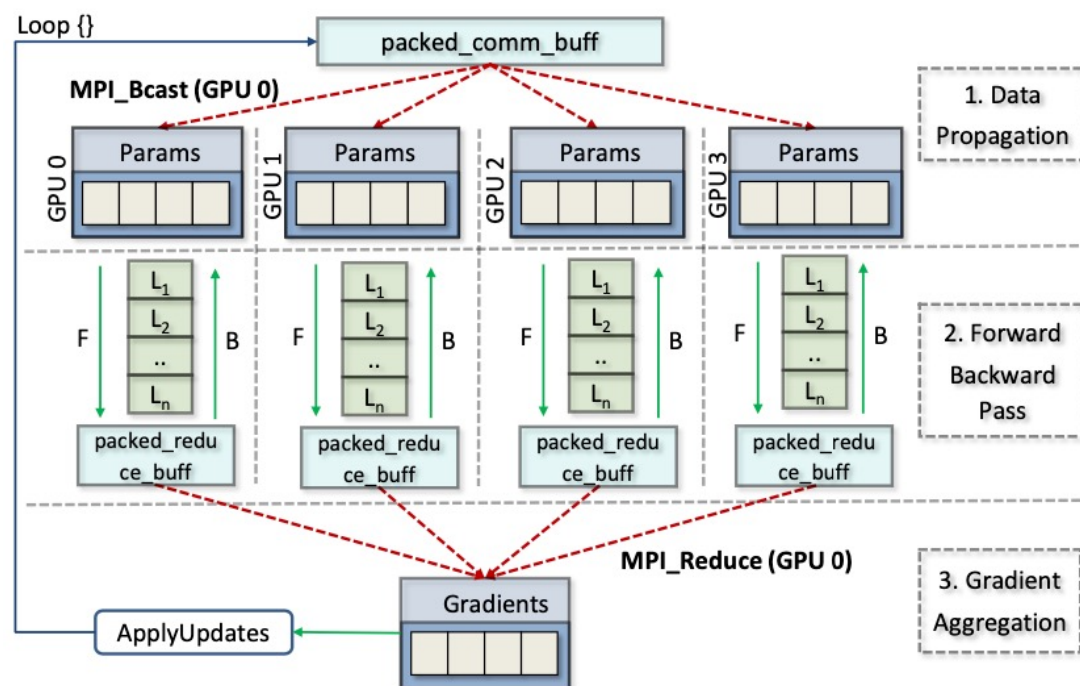
- PyTorch — MPI/Gloo/NCCL (dominant framework)
- TorchTitan — PyTorch-native LLM training at extreme scale
- DeepSpeed — ZeRO + CPU/NVMe offloading
- Megatron-LM — 3D parallelism (DP+TP+PP) for LLM pre-training
- JAX/XLA — Google framework for TPU and GPU training (Gemini, Gemma)



https://sea.ucar.edu/sites/default/files/DeepLearning_seaconf18.pdf

Data Parallel Deep Learning and MPI Collectives

- **Major MPI collectives involved in designing distributed frameworks:**
 - MPI_Bcast — required for DNN parameter exchange
 - MPI_Reduce — needed for gradient accumulation from multiple solvers
 - MPI_Allreduce — use just one AllReduce instead of Reduce + Broadcast
- **AllReduce pipelining with backprop:**
 - Start AllReduce on output-layer grads while inner layers still compute
 - Overlap of communication and compute = key to high scaling efficiency
 - This is exactly how PyTorch DDP achieves near-linear scaling

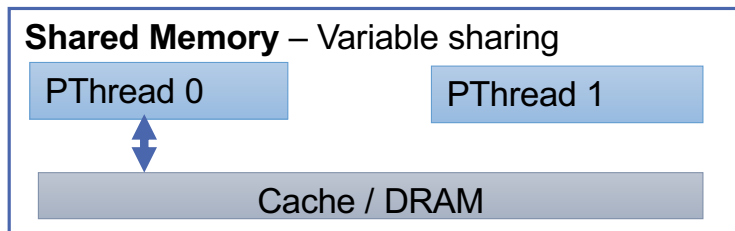


A. A. Awan, K. Hamidouche, J. M. Hashmi, and D. K. Panda, S-Caffe: Co-designing MPI Runtimes and Caffe for Scalable Deep Learning on Modern GPU Clusters. In *Proceedings of the 22nd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '17)*

Message Passing Interface

Distributed vs. Shared Memory Programming

- **Shared Memory:**
 - Access the same virtual address space
 - Pass data by reference (pointer)
 - Example: share a variable with pthreads
 - Tools: OpenMP, pthreads, CUDA shared mem
- Used within a single node (CPU cores or GPU threads)



- **Distributed Memory:**
 - Separate virtual address spaces
 - Cannot access shared data structures directly
 - Pass data by value (copy) via explicit message
 - Examples: process pipe, MPI send across network
- Used across nodes — MPI implements distributed memory programming

Distributed Memory - Pipe

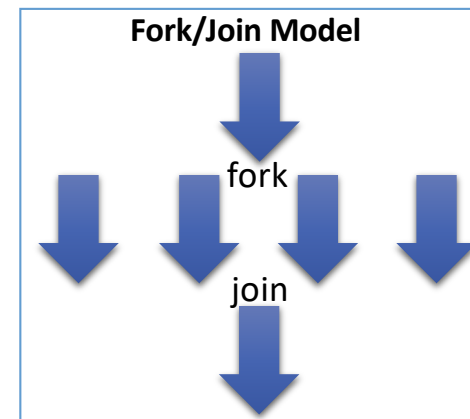
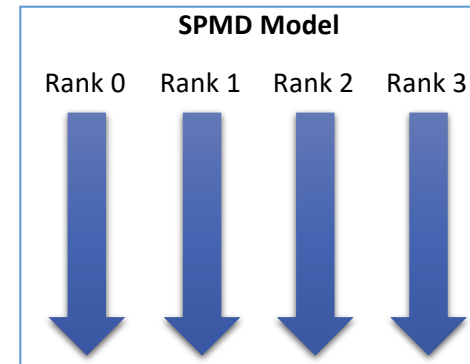


Distributed Memory - Message passing



SPMD vs Fork/Join

- **SPMD (Single Program Multiple Data):** execution starts in parallel (different from fork-join style)
 - **MPI** implements SPMD
 - 1 program executed by multiple processes (MPI ranks)
 - Programmers need to think in parallel from the beginning of the execution
 - **Static parallelism:** number of processes does not change
- **Fork/Join:** execution starts serial new processes or threads are created at fork operation
 - `pthread_create()/fork()` /OpenMP are examples of fork/join
 - **Dynamic parallelism:** number of processes can change at each fork
- MPI uses SPMD. The rank ID determines each process's behavior.

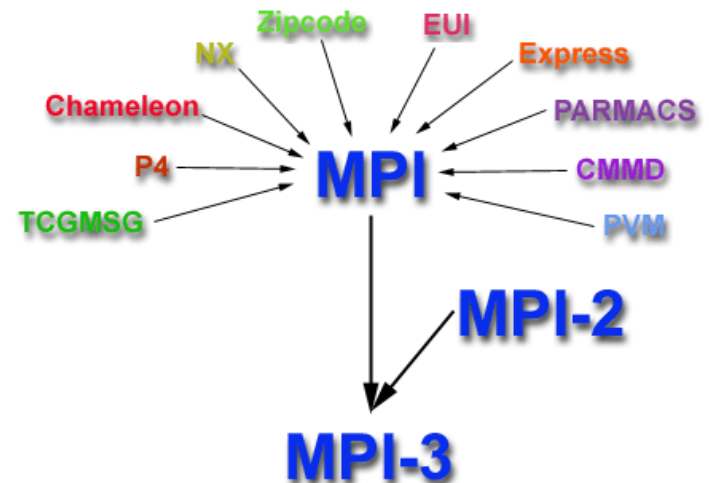


TCP vs MPI

	TCP Stack	MPI Stack
Addressing	IP address and port	Rank number in communicator
Communication	Point-to-point only	Point-to-point, collectives, one-sided
Orientation	Stream-oriented	Message-oriented
Data	Raw bytes/octetets	Typed messages (MPI_INT, MPI_FLOAT...)
Latency	High (kernel overhead)	Low (RDMA kernel bypass possible)
Network	Network independent	Network independent

Message Passing Interface

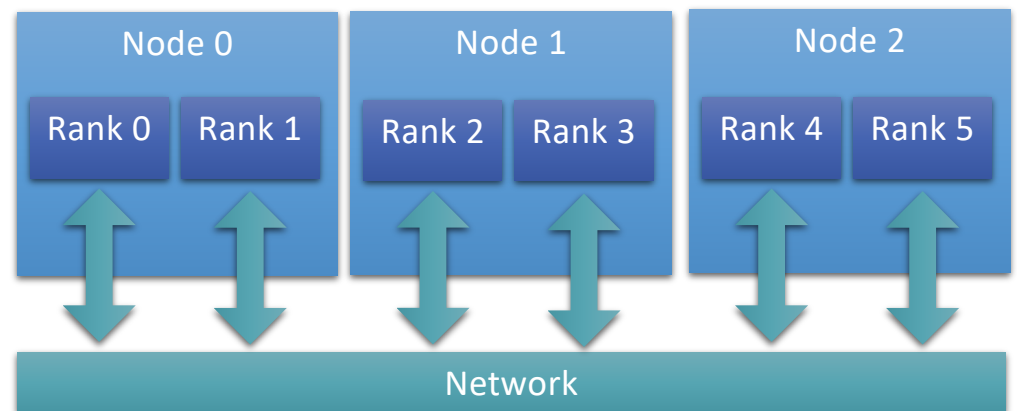
- **What is MPI?**
 - Standard specification for message-passing on large-scale parallel systems
- **Advantages:**
 - Standardization — portable across all major HPC platforms
 - Performance — tight RDMA and InfiniBand integration
 - Functionality — complete set from basic P2P to complex async collectives
- First version: MPI-1.0 in 1994
- **Current standard: MPI 4.1 (Nov 2023) · MPI 5.0 approved June 2025**



From: <https://computing.llnl.gov/tutorials/mpi/>

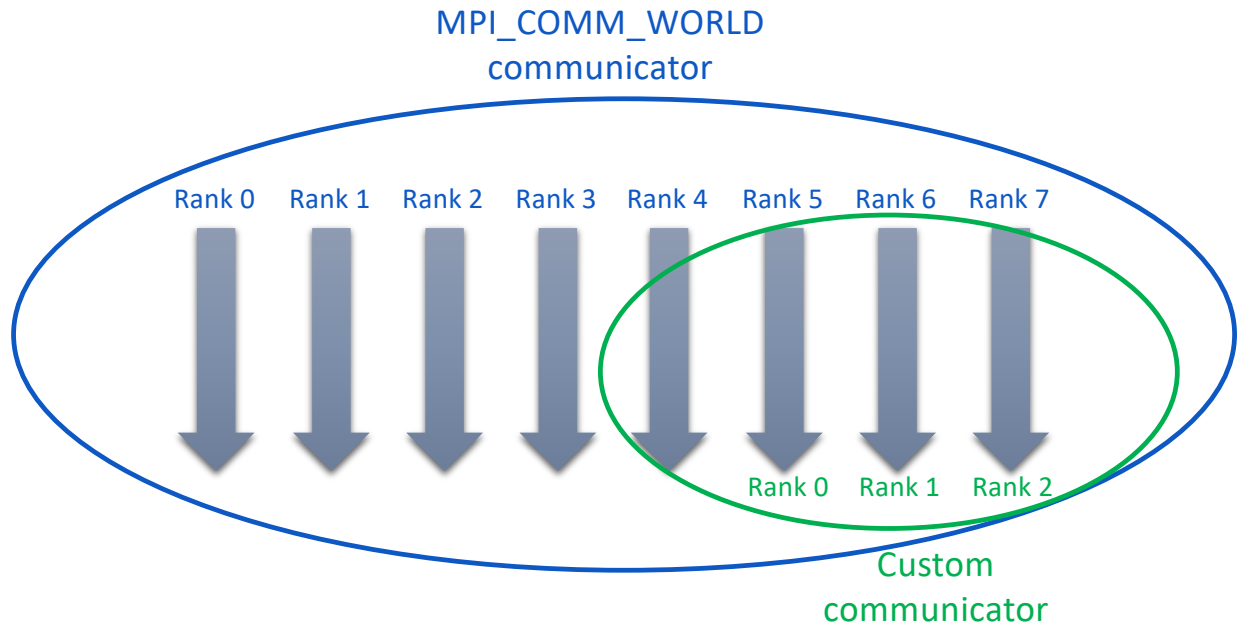
MPI – System components

- **Node:**
 - A single host machine on the network
- **Rank:**
 - A process executing the MPI program — usually one per core or GPU
- **MPI_COMM_WORLD:**
 - The default communicator that includes all ranks
 - Can be split into sub-communicators (DP group, TP group, PP group)
- MPI scales to thousands of nodes easily



MPI Programmer View

- Nodes are transparent to the programmer — only ranks matter
- **Communicator:**
 - A group of ranks that can communicate with each other
- **MPI_COMM_WORLD:**
 - The communicator that includes ALL ranks by default
- Custom communicators can be created with `MPI_Comm_split()`
- Critical for hybrid LLM parallelism: separate DP, TP, and PP communicators
- **DeviceMesh (PyTorch 2.3+)** is the modern abstraction replacing manual communicators



MPI Data Types

- MPI messages types and size is specified in advance
 - higher performance
 - less programming errors with types
- A message is always composed by one or more elements of MPI datatype size
- Definition of new types is possible
 - example: new data structures
- MPI_BYTE and also dynamic sizing is also available for special cases
- For complex types: MPI_Pack / MPI_Unpack for serialization
 - MPI provides support for sending messages of any size and any data type that can be described as a primitive type or a datatype derived from primitive types.
 - For types that cannot be transmitted in this way, such as linked lists or other objects that must be serialized, MPI provides a mechanism to “pack” (serialize) data into a buffer that can be transmitted via MPI, then “unpacked” (de-serialized) at the receiver’s end.

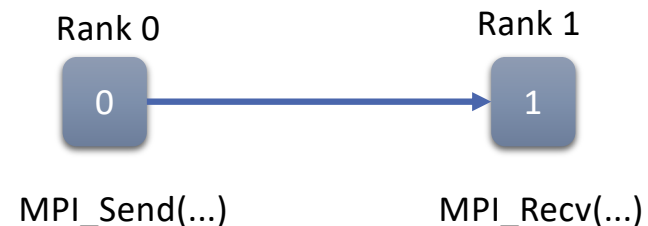
MPI datatype	C equivalent
MPI_SHORT	short int
MPI_INT	int
MPI_LONG	long int
MPI_LONG_LONG	long long int
MPI_UNSIGNED_CHAR	unsigned char
MPI_UNSIGNED_SHORT	unsigned short int
MPI_UNSIGNED	unsigned int
MPI_UNSIGNED_LONG	unsigned long int
MPI_UNSIGNED_LONG_LONG	unsigned long long int
MPI_FLOAT	float
MPI_DOUBLE	double
MPI_LONG_DOUBLE	long double
MPI_BYTE	char

Point-to-Point Communication

- **MPI_Send() and MPI_Recv():**
 - Can happen in any order — MPI runtime delivers the message
 - Destination based only on rank + communicator
 - Messages are typed, not just raw bytes
- **When MPI_Send() returns:**
 - Buffers can be reused
 - Does NOT mean message arrived
 - Does NOT mean it left the local node
- **Non-blocking: MPI_Isend / MPI_Irecv**
 - Process continues — wait later with MPI_Wait()

```
// Blocking send/receive
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
int number = -1;
if (rank == 0) {
    MPI_Send(&number, 1, MPI_INT,
             1, 0, MPI_COMM_WORLD);
} else if (rank == 1) {
    MPI_Recv(&number, 1, MPI_INT,
             0, 0, MPI_COMM_WORLD,
             MPI_STATUS_IGNORE);
}

// Non-blocking
MPI_Request req;
MPI_Isend(buf, 10, MPI_INT, right,
           123, MPI_COMM_WORLD, &req);
// ... do other work ...
MPI_Wait(&req, &status);
```



MPI_Status and MPI_Probe

- **Dynamic receiving with MPI_Probe and MPI_Status:**
- MPI_Recv can take MPI_ANY_SOURCE for the rank of the sender
- MPI_Recv can take MPI_ANY_TAG for the tag of the message
- MPI_Status structure is the only way to find out the actual sender and tag
- MPI_Get_count determines the actual number of elements received
- **MPI_Probe:**
 - Used to query the message size before receiving it
 - Blocks for a message with matching tag and sender
 - User can then call MPI_Recv to receive the actual message

Example using MPI_Status

```
const int MAX_NUMBERS = 100;
int numbers[MAX_NUMBERS];
int number_amount;
if (world_rank == 0) {
    // Pick a random amount of integers to send to process one
    srand(time(NULL));
    number_amount = (rand() / (float)RAND_MAX) * MAX_NUMBERS;

    // Send the amount of integers to process one
    MPI_Send(numbers, number_amount, MPI_INT, 1, 0, MPI_COMM_WORLD);
    printf("0 sent %d numbers to 1\n", number_amount);
} else if (world_rank == 1) {
    MPI_Status status;
    // Receive at most MAX_NUMBERS from process zero
    MPI_Recv(numbers, MAX_NUMBERS, MPI_INT, 0, 0, MPI_COMM_WORLD,
             &status);

    // After receiving the message, check the status to determine
    // how many numbers were actually received
    MPI_Get_count(&status, MPI_INT, &number_amount);

    // Print off the amount of numbers, and also print additional
    // information in the status object
    printf("1 received %d numbers from 0. Message source = %d, "
           "tag = %d\n",
           number_amount, status.MPI_SOURCE, status.MPI_TAG);
}
```

<https://mpitutorial.com/tutorials/dynamic-receiving-with-mpi-probe-and-mpi-status/>

MPI_Probe

- MPI_Probe is used to query the message size before receiving it.
- Similar to MPI_Recv, MPI_Probe will block for a message with a matching tag and sender. When the message is available, it will fill the status structure with information.
- The user can then use MPI_Recv to receive the actual message.

```
MPI_Probe(  
    int source,  
    int tag,  
    MPI_Comm comm,  
    MPI_Status* status)
```

Example with MPI_Probe

```
int number_amount;
if (world_rank == 0) {
    const int MAX_NUMBERS = 100;
    int numbers[MAX_NUMBERS];
    // Pick a random amount of integers to send to process one
    srand(time(NULL));
    number_amount = (rand() / (float)RAND_MAX) * MAX_NUMBERS;

    // Send the random amount of integers to process one
    MPI_Send(numbers, number_amount, MPI_INT, 1, 0, MPI_COMM_WORLD);
    printf("0 sent %d numbers to 1\n", number_amount);
} else if (world_rank == 1) {
    MPI_Status status;
    // Probe for an incoming message from process zero
    MPI_Probe(0, 0, MPI_COMM_WORLD, &status);

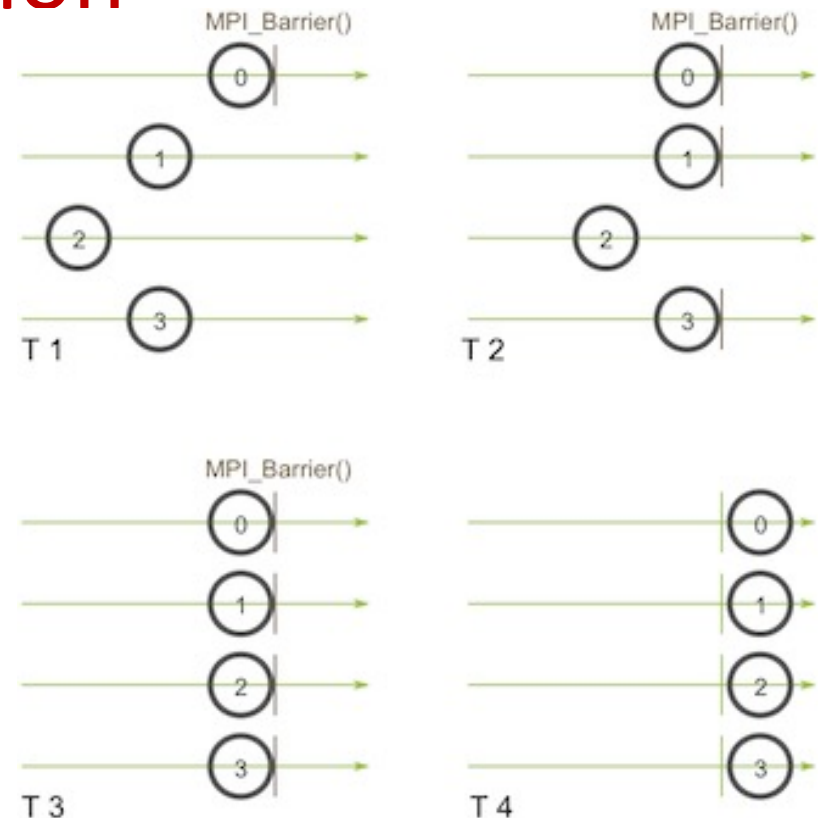
    // When probe returns, the status object has the size and other
    // attributes of the incoming message. Get the message size
    MPI_Get_count(&status, MPI_INT, &number_amount);

    // Allocate a buffer to hold the incoming numbers
    int* number_buf = (int*)malloc(sizeof(int) * number_amount);

    // Now receive the message with the allocated buffer
    MPI_Recv(number_buf, number_amount, MPI_INT, 0, 0,
             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("1 dynamically received %d numbers from 0.\n",
          number_amount);
    free(number_buf);
}
```

Barriers and Synchronization

- **MPI_Barrier():**
 - Synchronizes all ranks in a communicator
 - Each process waits until ALL processes reach the barrier before proceeding
 - Fundamental primitive for parallel programming correctness
 - Barrier implementation and latency are key to many algorithms' performance
- **Signature: MPI_Barrier(MPI_Comm communicator)**
- Some collectives can be hardware-accelerated:
 - SHARP in-network reduction offloads AllReduce to InfiniBand switch hardware

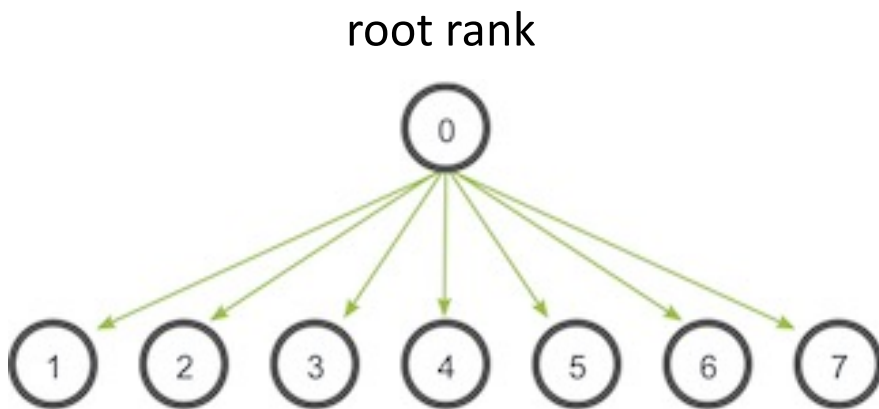


Collective Operations

- Collectives: communication patterns involving data exchange across multiple ranks
- **Types of collectives:**
 - Reduce, AllReduce — aggregate values from all ranks
 - Gather, AllGather — collect data from all ranks
 - Scatter — distribute data from one rank to all
 - AllToAll — each rank sends to each other rank
 - Broadcast — one rank sends same data to all
- Collectives represent the biggest value of MPI implementations
- Some collectives can be hardware-accelerated (SHARP on InfiniBand)
 - Offload MPI collectives into FPGA hardware
 - Hardware implementation of MPI Barrier on an FPGA cluster

Broadcast

- Root rank sends the SAME message to all receivers
- Message can contain multiple elements of type DataType
- **Required for DNN parameter exchange**

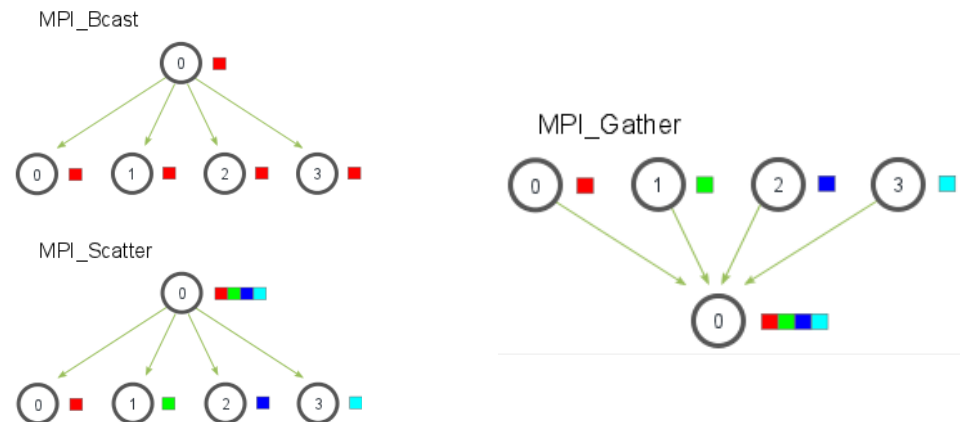


```
MPI_Bcast(  
    void* data,  
    int count,  
    MPI_Datatype datatype,  
    int root,  
    MPI_Comm communicator)  
  
// Example: broadcast model parameters  
if (rank == 0) {  
    // initialize model on rank 0  
}  
MPI_Bcast(model_params, num_params,  
          MPI_FLOAT, 0,  
          MPI_COMM_WORLD);  
// All ranks now have identical model
```

Scatter and Gather

- **Scatter:**
 - Root sends a DIFFERENT chunk to each receiver (in rank order)
 - MPI_Bcast sends the same data; MPI_Scatter sends different slices
- **Gather:**
 - Root collects a different message from each sender
 - Elements at root ordered by rank of the sender
- Combined: distribute work, gather results

```
MPI_Scatter(send_data, send_count,  
            send_datatype, recv_data,  
            recv_count, recv_datatype,  
            root, comm)  
  
MPI_Gather(send_data, send_count,  
           send_datatype, recv_data,  
           recv_count, recv_datatype,  
           root, comm)  
  
// recv_count = elements per process  
// NOT total across all processes
```

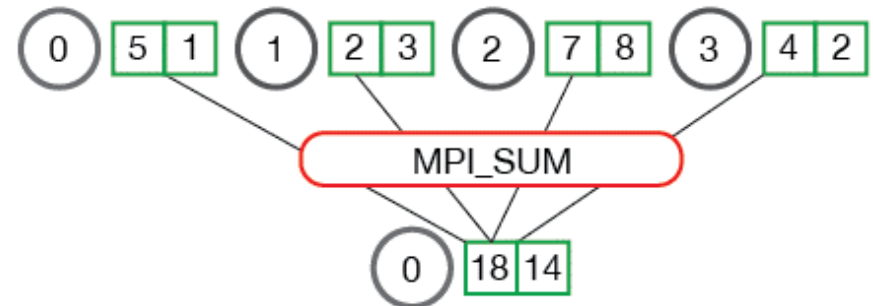
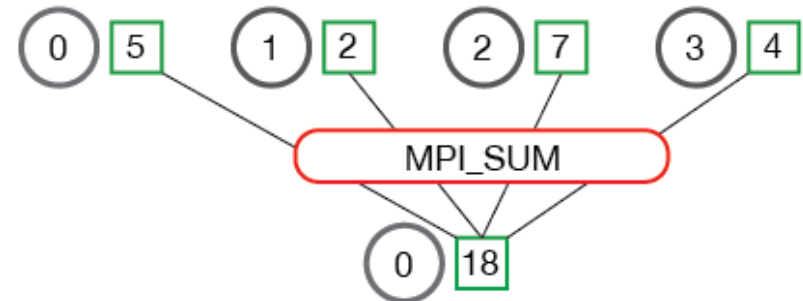


Reduce

- Reduces a set of numbers into a smaller set of numbers via a function
- In the picture MPI_SUM is used as reduce function
- Reduce can involve multiple elements per rank

```
MPI_Reduce(  
    void* send_data,  
    void* recv_data,  
    int count,  
    MPI_Datatype datatype,  
    MPI_Op op,  
    int root,  
    MPI_Comm communicator)
```

MPI_Reduce



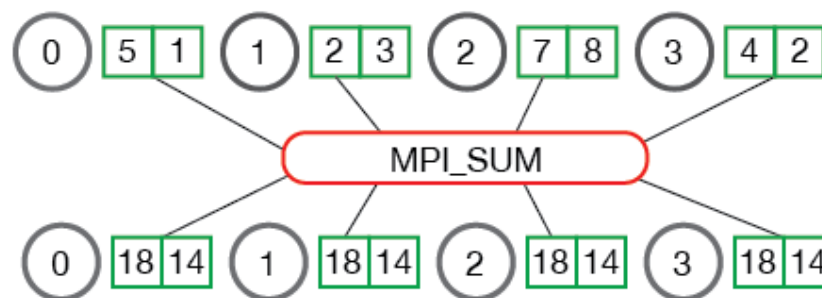
Reduce – Available Functions

- **MPI_MAX** - Returns the maximum element
- **MPI_MIN** - Returns the minimum element
- **MPI_SUM** - Sums the elements
- **MPI_PROD** - Multiplies all elements
- **MPI LAND** - Performs a logical AND across the elements
- **MPI_LOR** - Performs a logical OR across the elements
- **MPI_BAND** - Performs a bitwise AND across the bits of the elements
- **MPI_BOR** - Performs a bitwise OR across the bits of the elements
- **MPI_MAXLOC** - Returns the maximum value and the rank of the process that owns it
- **MPI_MINLOC** - Returns the minimum value and the rank of the process that owns it

AllReduce

- Reduces values and distributes the result to ALL ranks
- AllReduce = Reduce + Broadcast in one efficient operation
- The most important collective in deep learning:
 - Average gradients across all workers every training step
 - All workers update their local model identically after AllReduce
- Available ops: MPI_SUM, MPI_MAX, MPI_MIN, MPI_PROD, MPI_MAXLOC

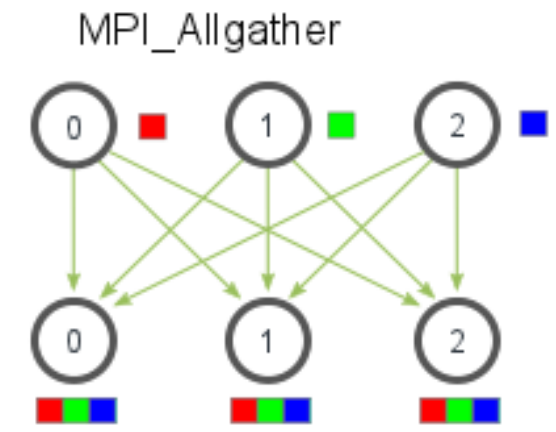
```
MPI_Allreduce(  
    void* send_data,  
    void* recv_data,  
    int count,  
    MPI_Datatype datatype,  
    MPI_Op op,  
    MPI_Comm communicator)  
  
// Gradient averaging example:  
float local_grad = compute_grad();  
float avg_grad;  
MPI_Allreduce(&local_grad, &avg_grad,  
              1, MPI_FLOAT, MPI_SUM,  
              MPI_COMM_WORLD);  
avg_grad /= world_size;
```



All Gather

- All Gather is composed of one gather for each rank
- Useful when all ranks need the result of the gather

```
MPI_Allgather(  
    void* send_data,  
    int send_count,  
    MPI_Datatype send_datatype,  
    void* recv_data,  
    int recv_count,  
    MPI_Datatype recv_datatype,  
    MPI_Comm communicator)
```



Average with Gather and Scatter

- Gather and Scatter example to compute the average of FLOAT numbers
- Ranks are in *world_rank*

```
if (world_rank == 0) {
    rand_nums = create_rand_nums(elements_per_proc * world_size);
}

// Create a buffer that will hold a subset of the random numbers
float *sub_rand_nums = malloc(sizeof(float) * elements_per_proc);

// Scatter the random numbers to all processes
MPI_Scatter(rand_nums, elements_per_proc, MPI_FLOAT, sub_rand_nums,
            elements_per_proc, MPI_FLOAT, 0, MPI_COMM_WORLD);

// Compute the average of your subset
float sub_avg = compute_avg(sub_rand_nums, elements_per_proc);
// Gather all partial averages down to the root process
float *sub_avgs = NULL;
if (world_rank == 0) {
    sub_avgs = malloc(sizeof(float) * world_size);
}
MPI_Gather(&sub_avg, 1, MPI_FLOAT, sub_avgs, 1, MPI_FLOAT, 0,
          MPI_COMM_WORLD);

// Compute the total average of all numbers.
if (world_rank == 0) {
    float avg = compute_avg(sub_avgs, world_size);
}
```

StdDev with All Reduce

- Using all reduce to compute the standard deviation of a set of numbers
- Each rank obtains the result

```
rand_nums = create_rand_nums(num_elements_per_proc);

// Sum the numbers locally
float local_sum = 0;
int i;
for (i = 0; i < num_elements_per_proc; i++) {
    local_sum += rand_nums[i];
}

// Reduce all of the local sums into the global sum in order to
// calculate the mean
float global_sum;
MPI_Allreduce(&local_sum, &global_sum, 1, MPI_FLOAT, MPI_SUM,
              MPI_COMM_WORLD);
float mean = global_sum / (num_elements_per_proc * world_size);

// Compute the local sum of the squared differences from the mean
float local_sq_diff = 0;
for (i = 0; i < num_elements_per_proc; i++) {
    local_sq_diff += (rand_nums[i] - mean) * (rand_nums[i] - mean);
}

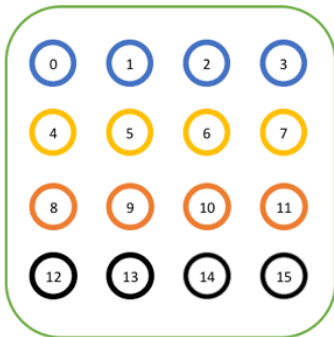
// Reduce the global sum of the squared differences to the root
// process and print off the answer
float global_sq_diff;
MPI_Reduce(&local_sq_diff, &global_sq_diff, 1, MPI_FLOAT, MPI_SUM, 0,
           MPI_COMM_WORLD);

// The standard deviation is the square root of the mean of the
// squared differences.
if (world_rank == 0) {
    float stddev = sqrt(global_sq_diff /
                        (num_elements_per_proc * world_size));
    printf("Mean - %f, Standard deviation = %f\n", mean, stddev);
}
```

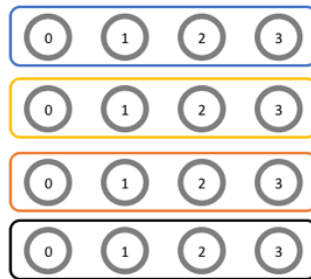
Multiple Communicators

- MPI_COMM_WORLD can be split into sub-communicators
- Rank numbers scoped to their own communicator
- **Critical for 4D parallelism in LLM training:**
 - Data parallel group (dp_comm)
 - Tensor parallel group within node (tp_comm)
 - Pipeline parallel group (pp_comm)
- PyTorch DeviceMesh is the modern version of this concept

MPI_COMM_WORLD
communicator



Multiple
communicators



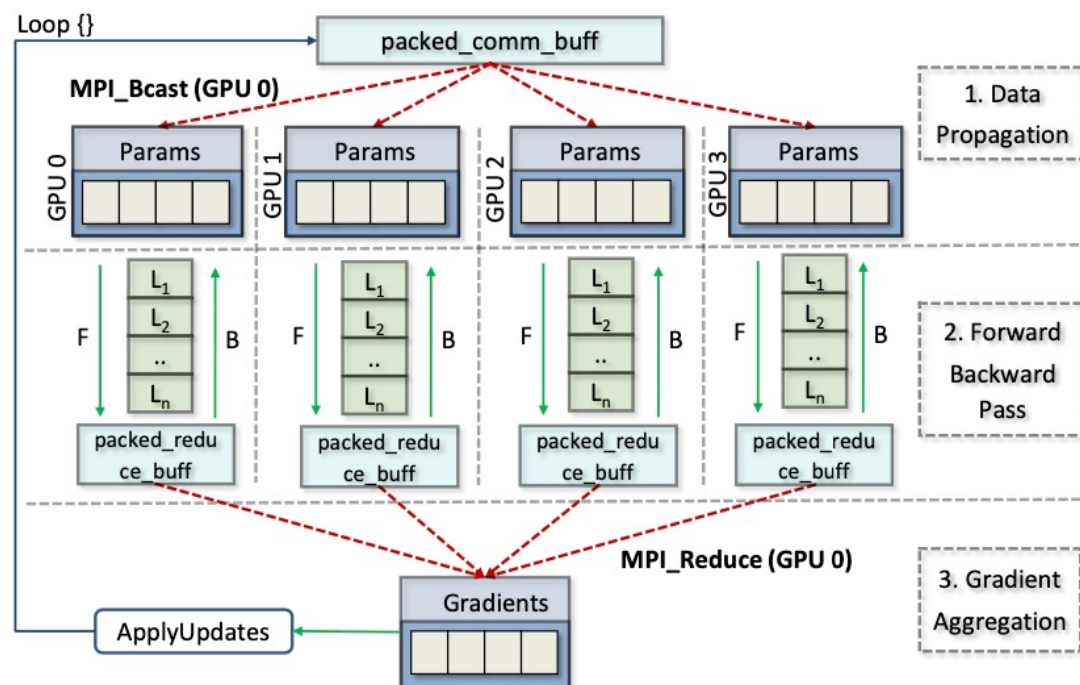
HPML

```
int world_rank, world_size;
MPI_Comm_rank(MPI_COMM_WORLD,
               &world_rank);
MPI_Comm_size(MPI_COMM_WORLD,
               &world_size);
// Group ranks into rows of 4
int color = world_rank / 4;
MPI_Comm row_comm;
MPI_Comm_split(MPI_COMM_WORLD,
               color, world_rank,
               &row_comm);

int row_rank, row_size;
MPI_Comm_rank(row_comm, &row_rank);
MPI_Comm_size(row_comm, &row_size);
printf("WORLD: %d/%d ROW: %d/%d\n",
       world_rank, world_size,
       row_rank, row_size);
MPI_Comm_free(&row_comm);
```

Data Parallel Deep Learning and MPI Collectives

- Major **MPI Collectives** involved in Designing distributed frameworks
- **MPI_Bcast** – required for DNN parameter exchange
- **MPI_Reduce** – needed for gradient accumulation from multiple solvers
- **MPI_Allreduce** – use just one Allreduce instead of Reduce and Broadcast



A. A. Awan, K. Hamidouche, J. M. Hashmi, and D. K. Panda, S-Caffe: Co-designing MPI Runtimes and Caffe for Scalable Deep Learning on Modern GPU Clusters. In *Proceedings of the 22nd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '17)*

Parallel and Distributed Deep Learning Algorithms

Motivation

Train a Video Model (TSM) on 660 Hours of Videos

Models	Training Time	Accuracy
1 SUMMIT Nodes (6 GPUs)	49h 50min	74.1%
128 SUMMIT Nodes (768 GPUs)	28min	74.1%
256 SUMMIT Nodes (1536 GPUs)	14min (211x)	74.0%

- Speed up the training by 200x, from 2 days to 14 **minutes**.
- **Model setup**: 8-frame ResNet-50 TSM for the video recognition task
- **Dataset**: Kinetics (240k training videos) x 100 epoch



Data obtained from: <https://efficientml.ai>

DL Parallelism Approaches

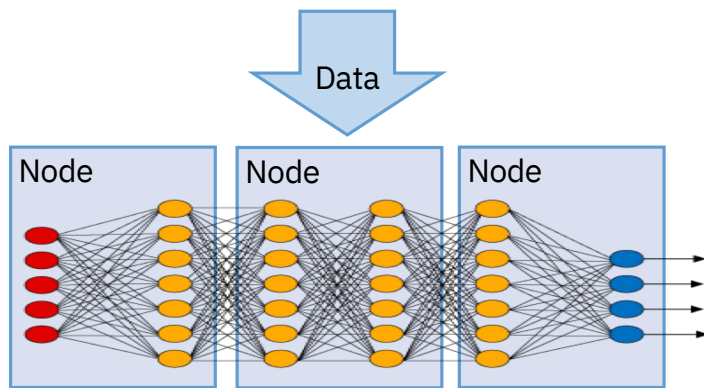
- **Model Parallelism**
 - Split model across multiple compute engines (GPUs)
- **Pipelining**
 - Use layers as pipeline to keep all compute engines busy
- **Data Parallelism**
 - Split data across multiple compute engines
- **Hybrid Parallelism**
 - Use multiple approaches together (DP + TP + PP + SP)
- **Large-Scale Training Approaches**
 - ZeRO-1/2/3 and FSDP2 — memory-efficient sharding across GPUs
 - Tensor Parallelism — split weight matrices within a layer (NEW)

DL Parallelism Approaches

- **Model Parallelism**
 - Split model across multiple compute engines (GPUs)
- **Pipelining**
 - Use layers as pipeline to keep all compute engines busy
- **Data Parallelism**
 - Split data across multiple compute engines
- **Hybrid Parallelism**
 - Use multiple approaches together (DP + TP + PP + SP)
- **Large-Scale Training Approaches**
 - ZeRO-1/2/3 and FSDP2 — memory-efficient sharding across GPUs
 - Tensor Parallelism — split weight matrices within a layer (NEW)

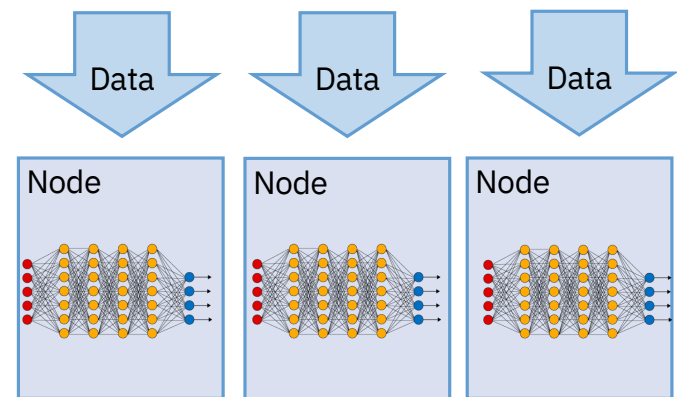
Key Parallelization Approaches

Model Parallelism



- Parameters are distributed across nodes
- Mini-batch has to be copied across all processors
- **Backpropagation requires all-to-all communication every layer**

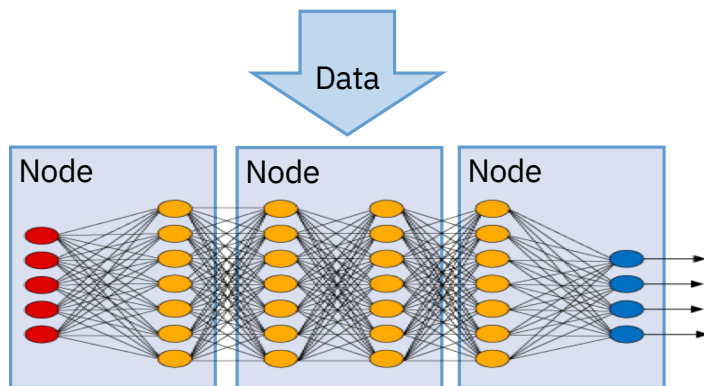
Data Parallelism



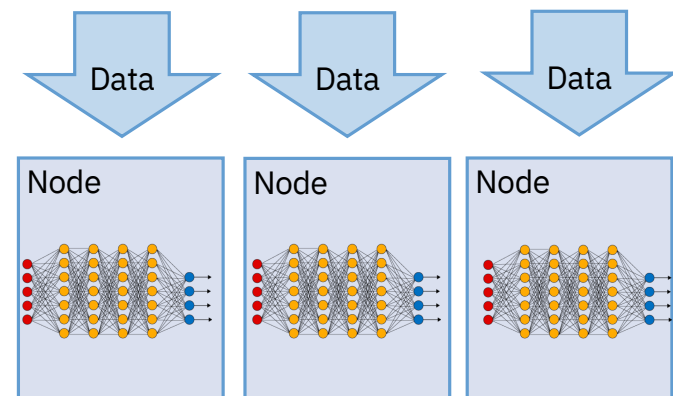
- Simple and efficient solution, easy to implement
- **Duplicate parameters across all nodes**
- **Most prevalent method in deep learning**

Key Parallelization Approaches

Model Parallelism



Data Parallelism



Tensor Parallelism (NEW)

- Parameters are split across GPUs
- Mini-batch is processed by all processors
- Backpropagation requires communication every layer**
- Split individual weight matrices (W) within a layer across GPUs
- Column-parallel: W split along output dim $\rightarrow$ each GPU holds columns $\rightarrow$ AllReduce
- Row-parallel: W split along input dim $\rightarrow$ partial outputs $\rightarrow$ Reduce-Scatter
- Most prevalent method in deep learning**

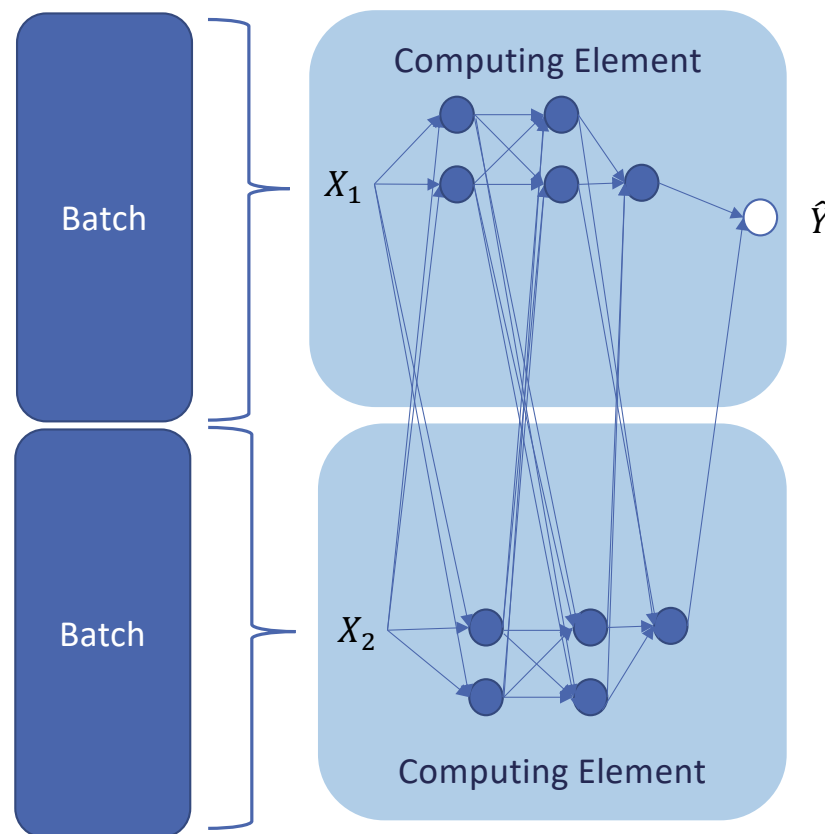
ML Model Parallelism

- **Model Parallelism:**

1. Divide the model into parts
2. Each computing element (CPU core or GPU thread) receives its part of the model
3. Send the same batch to the computing elements
4. Activations flow between GPUs at layer boundaries
5. Model merges at the end

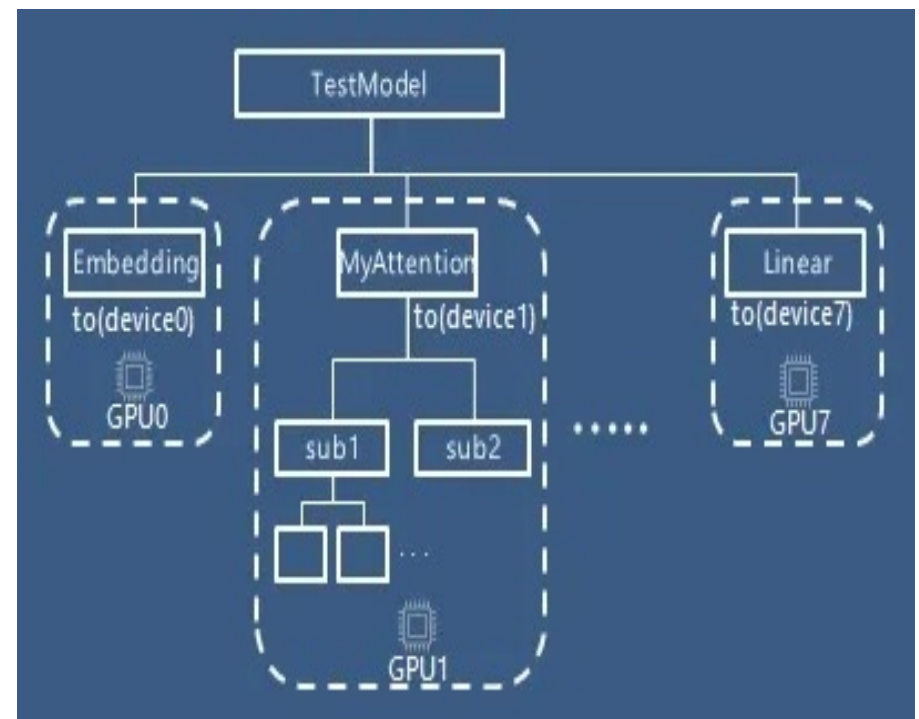
- **Characteristics:**

- **The latency of communication between the computing elements degrades performance**
- Harder to obtain performance if elements are far apart (use NVLink)
- Used among GPU threads or CPU cores/threads within a node



Model Parallelism — Layer Assignment to GPUs

- **Layers assigned to different devices:**
 - Embedding layer → GPU 0 (cuda:0)
 - Attention layers 1–6 → GPUs 1–6
 - Language model head → GPU 7 (cuda:7)
- **Key implementation detail:**
 - Each layer's `.to('cuda:N')` call moves parameters to that GPU
 - Forward pass: activations transferred at each layer boundary
 - Challenge: only one GPU active at a time — poor utilization



Source: Matsuzaki, T. (2023). Implement Model Parallelism in LLMs. tsmatz.wordpress.com/2023/09/21/model-parallelism/

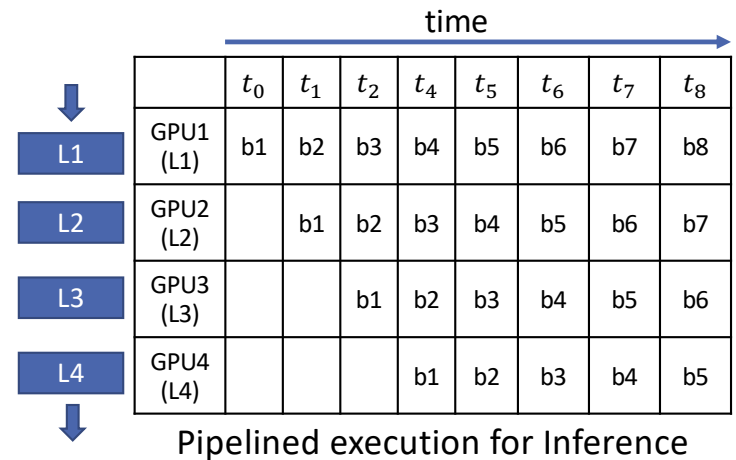
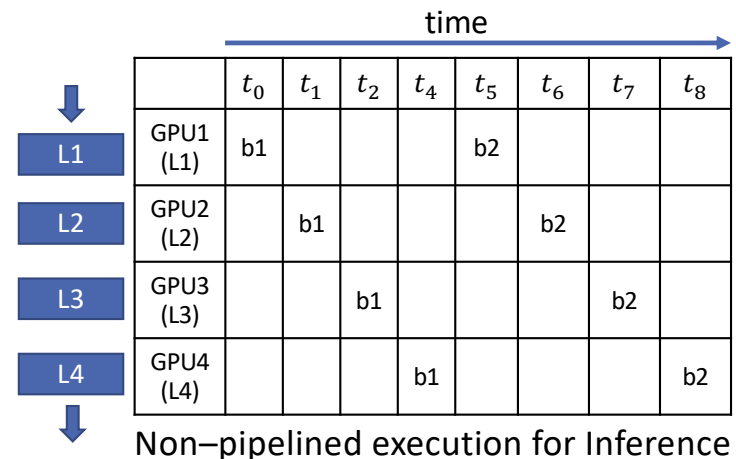
DL Parallelism Approaches

- Model Parallelism
 - Split model across multiple compute engines (GPUs)
- **Pipelining**
 - **Use layers as pipeline to keep all compute engines busy**
- Data Parallelism
 - Split data across multiple compute engines
- Hybrid Parallelism
 - Use multiple approaches together (DP + TP + PP + SP)
- Large-Scale Training Approaches
 - ZeRO-1/2/3 and FSDP2 — memory-efficient sharding across GPUs
 - Tensor Parallelism — split weight matrices within a layer (NEW)

DL Pipelining

- Pipelining approach:
 - Split layers among compute engines (GPUs or workers)
 - Each minibatch b (or sample s) goes from one compute engine to the next one: *no need to wait for next one to exit the pipeline*
- It is a form of **Model Parallelism**
- Pipelining performance
 - Ideal pipelining speedup (*number of pipeline stages*)

$$S_{time}(stage\ time) = \frac{time\ without\ pipeline}{number\ of\ pipeline\ stages}$$
 - Speedup is higher for deeper networks
 - Ideal pipelining never reached because of “bubbles” that cause idle CPUs
 - SGD pipeline bubble:
 - Before weights update, all batches need to have completed forward (otherwise accept **staleness**)

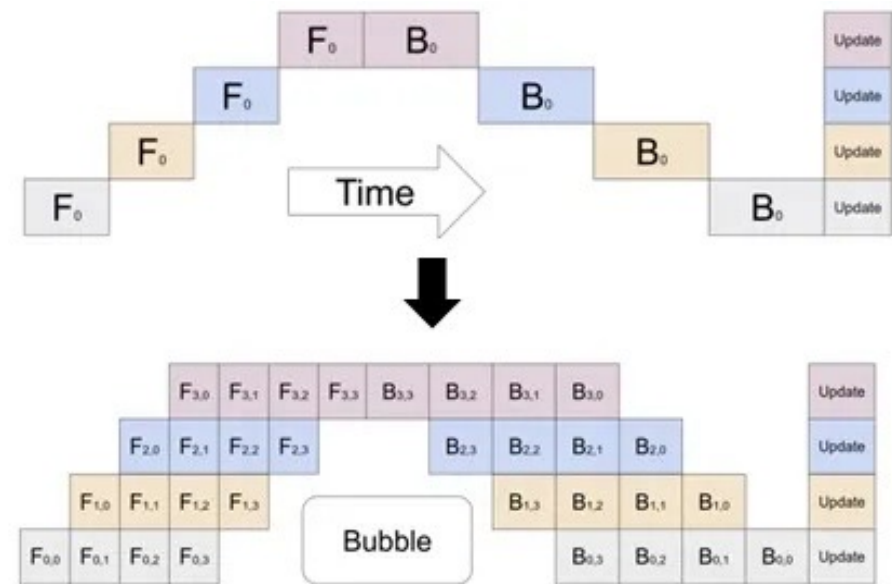


DL Pipeline variants

- **GPipe** — split global batch into M microbatches, $O(M)$ memory demand
- **Interleaved (1F1B)** — backward executed as soon as possible, more memory-efficient
- **Zero-Bubble**: defers weight gradient to fill idle slots, ~50% bubble reduction
- **Zero-Bubble pipeline (2024)** integrated in TorchTitan and Megatron-LM

DL Pipelining — GPipe Microbatch Scheduling

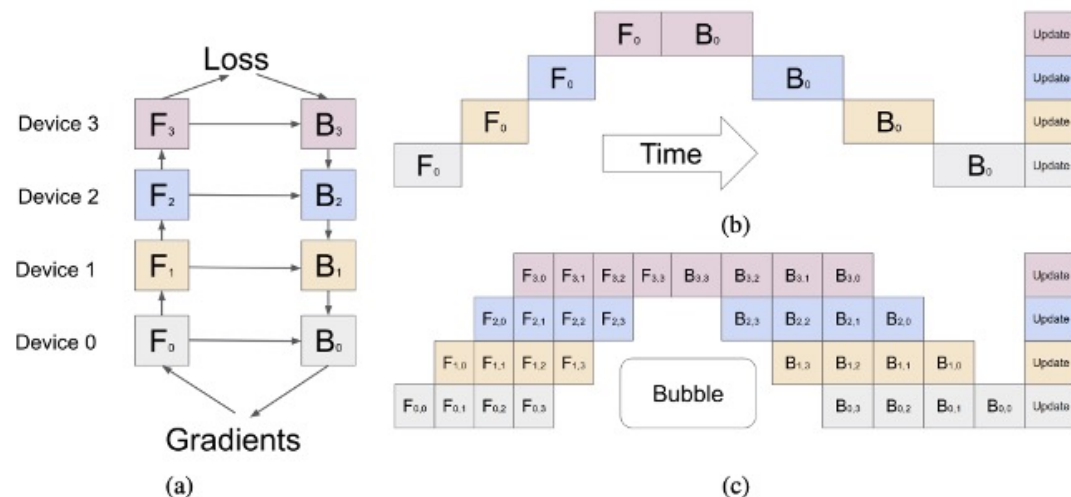
- **Top: Sequential execution**
 - Only one GPU active at a time
 - All other GPUs idle — wasteful
- **Bottom: Pipelined execution**
 - Global batch split into microbatches
 - F = Forward pass, B = Backward pass
 - Multiple GPUs active simultaneously
 - Pipeline bubble = idle time at start/end
- GPipe achieves near-ideal speedup for deep networks



F = Forward Pass, B = Backward Pass

[GPipe: Easy Scaling with Micro-Batch Pipeline Parallelism](https://github.com/tensorflow/lingvo/blob/master/lingvo/core/gpipe.py)
<https://github.com/tensorflow/lingvo/blob/master/lingvo/core/gpipe.py>
<https://blog.research.google/2019/03/introducing-gpipe-open-source-library.html>

GPipe Pipelining



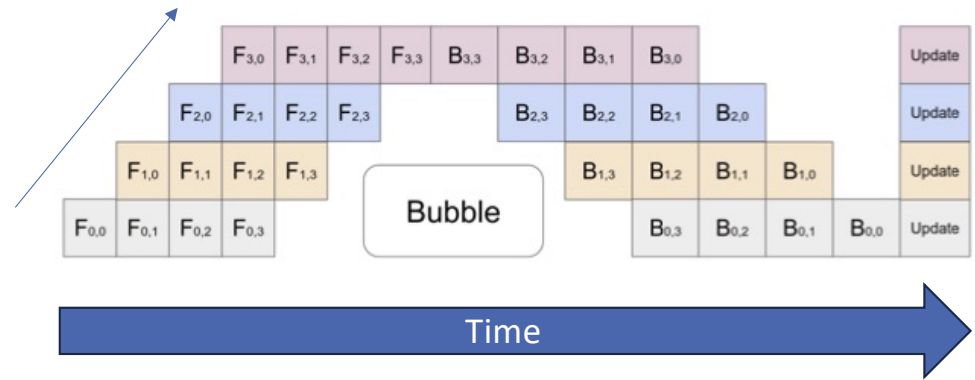
Different GPUs or workers work on different layers

- Gpipe is an open-source pipeline parallelism library that allows scaling any network that can be expressed as a sequence of layers.
- Split global batch into multiple micro-batches and inject them into the pipeline concurrently
- It is not memory-friendly and will not scale well with large batches. The activations produced by forward tasks have to be kept for all micro-batches until corresponding backward tasks start, so the memory demand is proportional ($O(M)$) to the number of concurrently scheduled micro-batches (M).

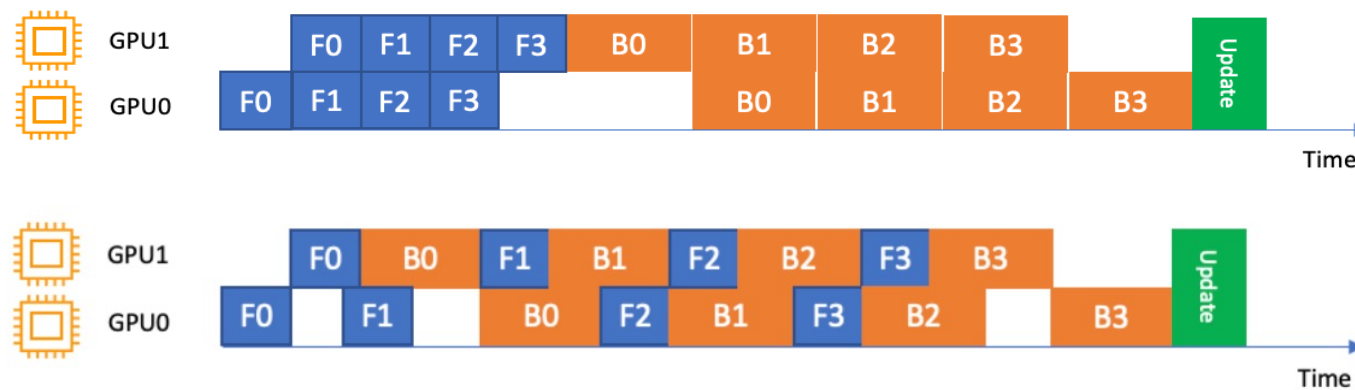
[GPipe: Easy Scaling with Micro-Batch Pipeline Parallelism](https://github.com/tensorflow/lingvo/blob/master/lingvo/core/gpipe.py)
<https://github.com/tensorflow/lingvo/blob/master/lingvo/core/gpipe.py>
<https://blog.research.google/2019/03/introducing-gpipe-open-source-library.html>

GPipe Pipelining – How does it work?

- Integrate data and model parallelism.
- Key idea:
 - Divide the data mini-batch into micro-batches
 - Different workers (accelerators) work on different micro-batches at the same time
 - The workers sync-up at the end. Gradients are updated when the entire mini-batch is processed



Interleaved Pipeline



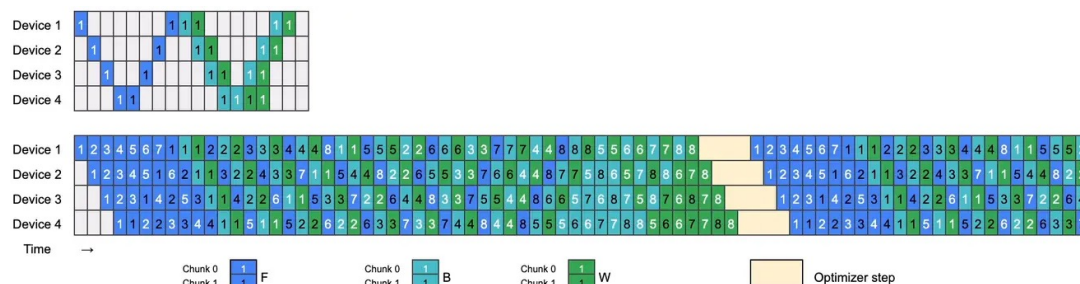
Simple Pipeline

Inter-leaved Pipeline

- Backward execution of the microbatches is prioritized whenever possible.
- Allows quicker release of the memory used for activations, using memory more efficiently
- It also allows for scaling the number of microbatches higher, reducing the idle time of the GPUs
- At steady-state, each device alternates between running forward and backward passes
 - Backward pass of one microbatch may run before the forward pass of another microbatch finishes

<https://docs.aws.amazon.com/sagemaker/latest/dg/model-parallel-core-features-pipeline-parallelism.html>

Pipeline Variants: Interleaved & Zero-Bubble



- **Interleaved Pipeline (1F1B):**

- Backward execution of micro-batches prioritized whenever possible
- Allows quicker release of activation memory — more memory efficient (If you delay backward too long, you must keep many activations in memory)
- At steady state: device alternates forward and backward passes

Zero-Bubble Pipeline Parallelism (ZB-H1/H2)

- Key idea: split weight gradient (W) from input gradient (B) computation
- W-grad deferred and scheduled to fill slots that would otherwise be idle bubbles
- ZB-H1: ~50% bubble reduction vs interleaved 1F1B with same memory cost
- ZB-H2: near-zero bubble time with slightly higher memory usage
- **Status: integrated in TorchTitan and Megatron-LM (2024–2025)**

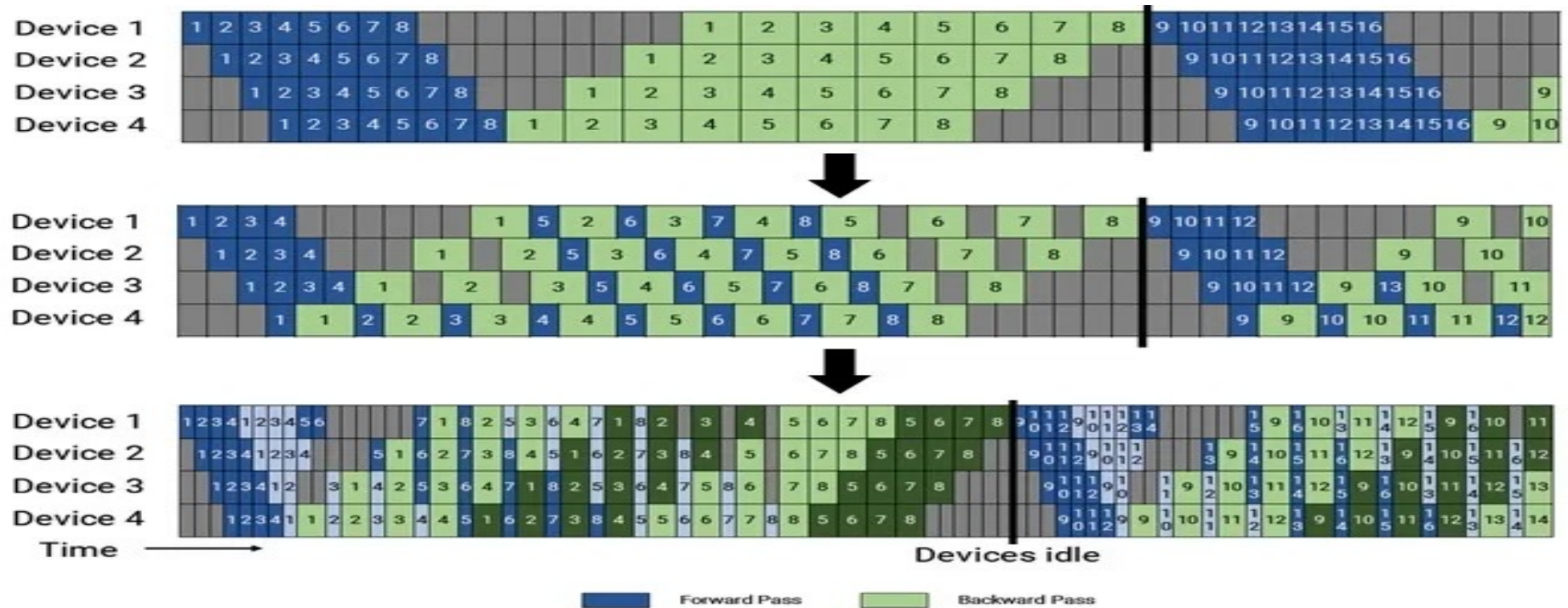
Conceptual algorithm for zero-bubble scheduling

For each stage and each micro-batch:

1. Do forward pass F
 2. When backward begins, split backward into:
 - B: gradient to send upstream
 - W: local weight gradient
 3. Prioritize B so the pipeline keeps moving
 4. Place W into idle windows where possible
 5. Delay optimizer-related synchronization if needed to preserve the no-bubble goal
 - The original zero-bubble work also discusses bypassing optimizer-step synchronization as part of achieving true zero-bubble behavior under synchronous training semantics.
- So the schedule is no longer just:
 - F, B, F, B, ...
 - It becomes more like:
 - F, B-critical, maybe other stage's F/B-critical, then W inserted opportunistically

That is a much more flexible scheduling problem.

Pipeline Scheduling: Simple vs. Interleaved vs. Optimized



Source: Narayanan et al. (2021). Megatron-LM. SC'21. Via Matsuzaki (2023) tsmatz.wordpress.com — Top: simple 1F1B. Middle: interleaved. Bottom: further optimized. Gray = idle (bubble).
HPML

DL Parallelism Approaches

- **Model Parallelism**
 - split model across multiple compute engines
- **Pipelining**
 - use layers as pipeline to keep all compute engines busy
- **Data Parallelism**
 - split data across multiple compute engines
- **Hybrid Parallelism**
 - use multiple approaches together
- **Large-Scale Training Parallelism Approaches**
 - FSDP

DL Parallelism Approaches

- Model Parallelism
 - Split model across multiple compute engines (GPUs)
- Pipelining
 - Use layers as pipeline to keep all compute engines busy
- **Data Parallelism**
 - **Split data across multiple compute engines**
- **Hybrid Parallelism**
 - **Use multiple approaches together (DP + TP + PP + SP)**
- Large-Scale Training Approaches
 - ZeRO-1/2/3 and FSDP2 — memory-efficient sharding across GPUs
 - Tensor Parallelism — split weight matrices within a layer (NEW)

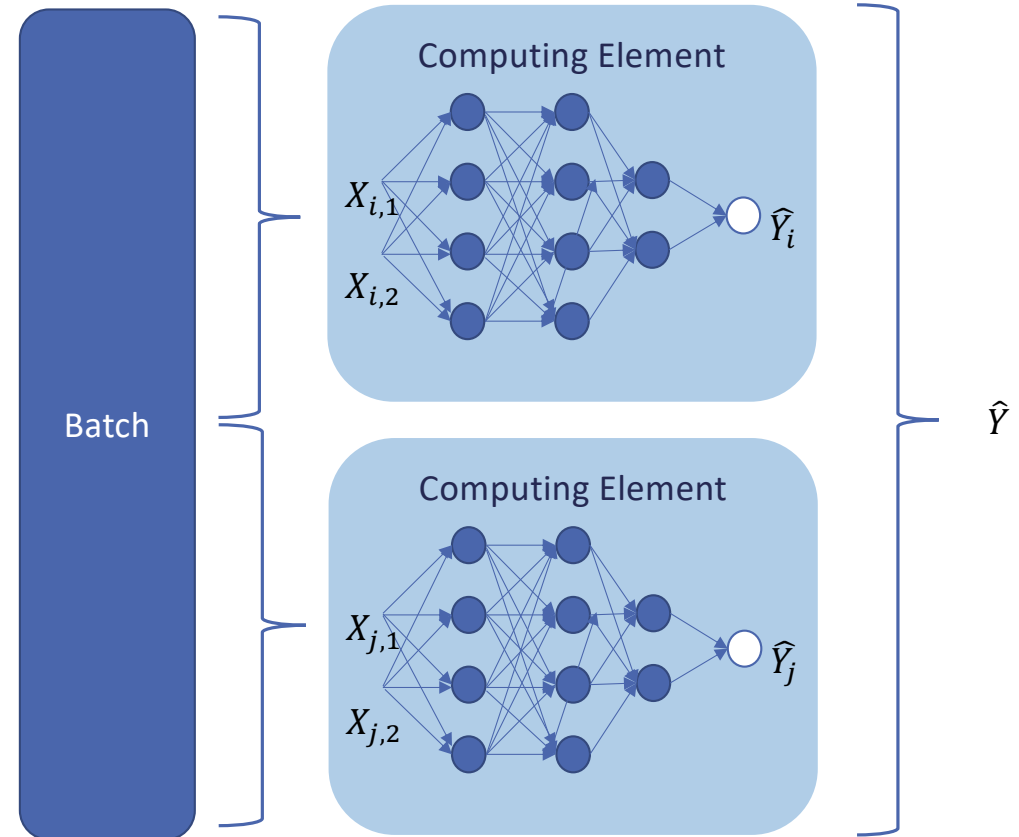
ML Data Parallelism

- **Data Parallelism:**

1. replicate model on each computing element (CPU core, GPU thread, or Cluster node)
2. Each computing element gets a portion of the batch
3. Each computing element trains/infers in parallel on the model
4. Gather the results at the end.

- **Characteristics:**

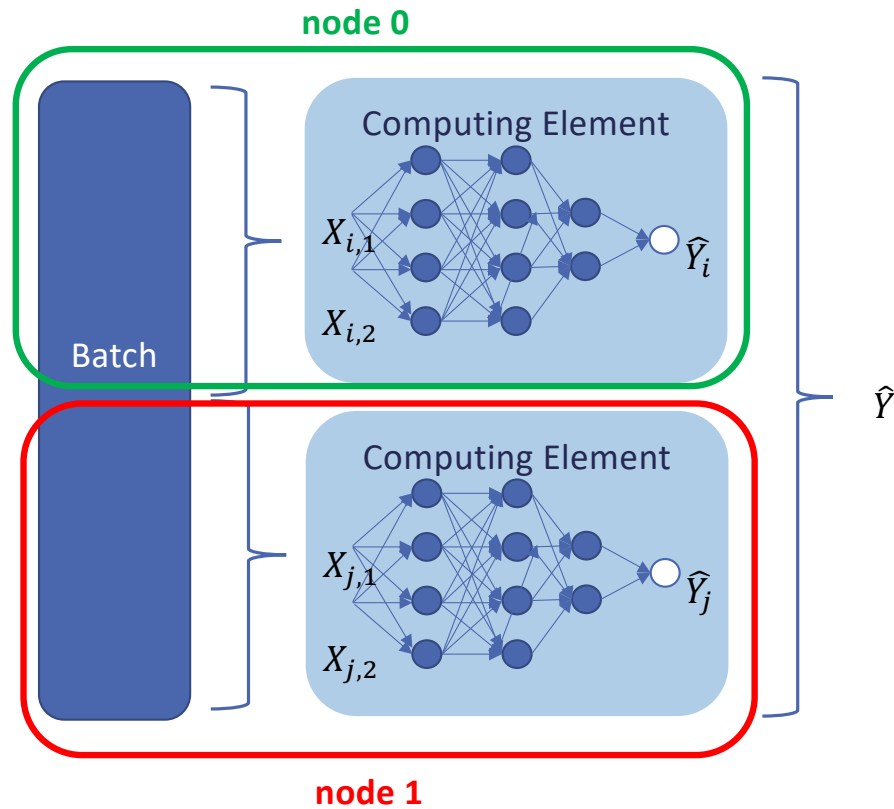
- Used among GPUs devices or among cluster nodes
- In PyTorch: DataParallel
- Can be also implemented using different batches instead of parts of one batch (typically among nodes in a cluster)



Distributed Data Parallelism

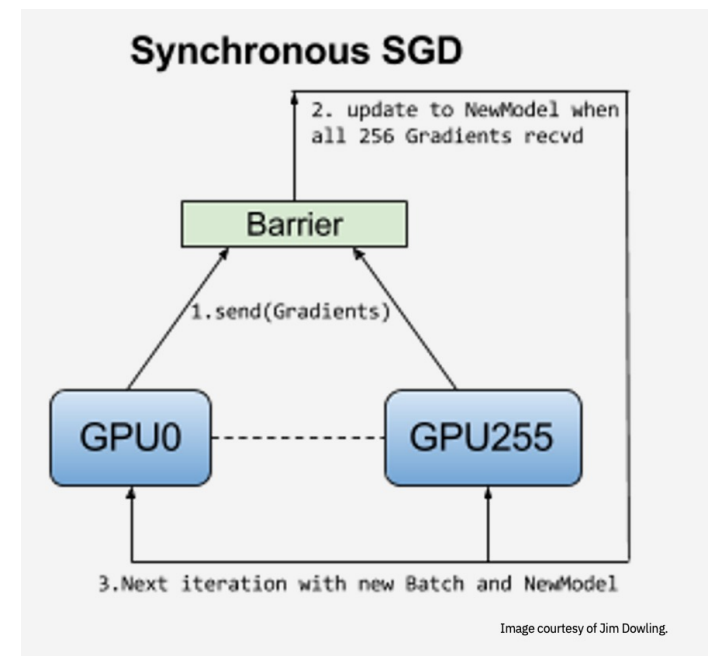
- **Distributed Data Parallelism**

- Computing elements belong to different nodes and processes
- Requirements:
 - Parallel-Distributed SGD algorithm
 - Communication infrastructure
 - In Pytorch: `DistributedDataParallel`



Synchronous Distributed SGD

- In the synchronous setting, all replicas average all of their gradients at every timestep (minibatch)
 - Doing so, we're effectively multiplying the batch size M by the number of replicas R
- This has several advantages:
 - The computation is completely deterministic
 - We can work with fairly large models and large batch sizes even on memory-limited GPUs
 - It's very simple to implement, and easy to debug and analyze



Synchronous Distributed SGD

- Distributed data parallel SGD is basically the classic minibatch SGD with lines 3 and 7 modified to read/write weights from/to a parameter store which may be centralized or decentralized
- This incurs an overhead on the overall system

Algorithm 2 Minibatch Stochastic Gradient Descent with Backpropagation

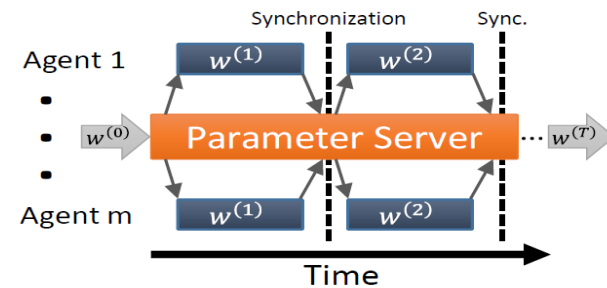
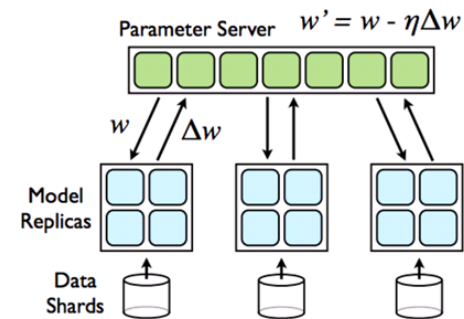
```
1: for  $t = 0$  to  $\frac{|S|}{B} \cdot epochs$  do
2:    $\vec{z} \leftarrow$  Sample  $B$  elements from  $S$                                  $\triangleright$  Obtain samples from dataset
3:    $w_{mb} \leftarrow w^{(t)}$                                                $\triangleright$  Load parameters
4:    $f \leftarrow \ell(w_{mb}, \vec{z}, h(\vec{z}))$                                  $\triangleright$  Compute forward evaluation
5:    $g_{mb} \leftarrow \nabla \ell(w_{mb}, f)$                                  $\triangleright$  Compute gradient using backpropagation
6:    $\Delta w \leftarrow u(g_{mb}, w^{(0, \dots, t)}, t)$                      $\triangleright$  Weight update rule
7:    $w^{(t+1)} \leftarrow w_{mb} + \Delta w$                                  $\triangleright$  Store parameters
8: end for
```

- S : the full training dataset
- B : minibatch size
- t : training step or iteration
- $\vec{z}$: sampled minibatch data
- w : model weights
- $w_{mb}^{(t)}$: weights used for this minibatch at iteration t
- f : loss value
- g_{mb} : gradient computed on the minibatch
- Δw : weight update
- $\mu(\cdot)$: update rule, such as SGD or Adam

Sample data $\rightarrow$ forward pass $\rightarrow$ loss $\rightarrow$ backprop $\rightarrow$ optimizer update $\rightarrow$ new weights $\rightarrow$ repeat

Synchronous Distributed SGD – (Parameter Server)

1. MiniBatch is partitioned among workers and model is replicated on each worker
2. A **Parameter server** contains copy of the model
3. Each worker executes forward and backward propagation on its minibatch portion
4. Gradients are sent to the parameter server that computes the update
5. The workers receive updated models from the parameter server

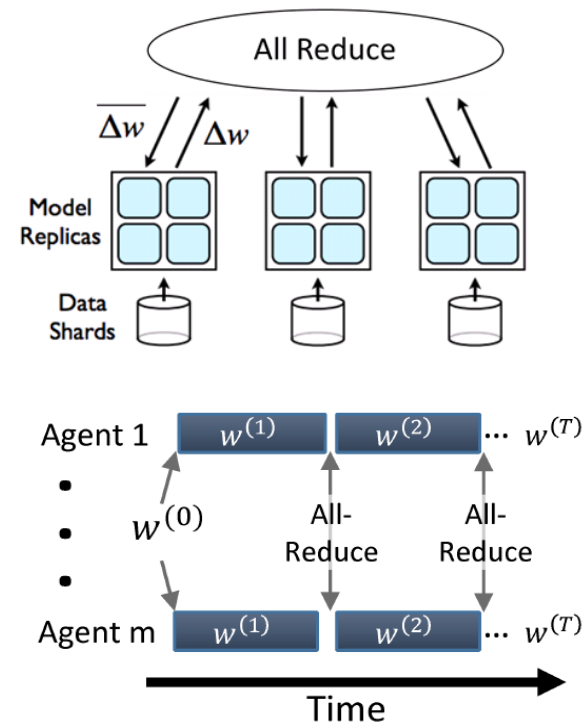


Synchronous Centralized Distributed SGD

from: <https://arxiv.org/abs/1802.09941>

Synchronous Distributed SGD – (All Reduce)

- MiniBatch partitioned among workers; model replicated on each
- Each worker executes forward and backward on its mini-batch portion
- Gradients averaged with AllReduce operation across all workers
- All workers update with identical averaged gradients
- **Communication cost per GPU:**
 - Ring AllReduce: $2N(P-1)/P$ — independent of P for large P
 - VS Parameter Server: $2N(P-1)$ at PS — grows with P
 - Ring AllReduce is significantly more bandwidth-efficient at scale
- **Used in: PyTorch DDP, Horovod, FSDP2, Megatron-LM**



Synchronous Decentralized Distributed SGD
from: <https://arxiv.org/abs/1802.09941>

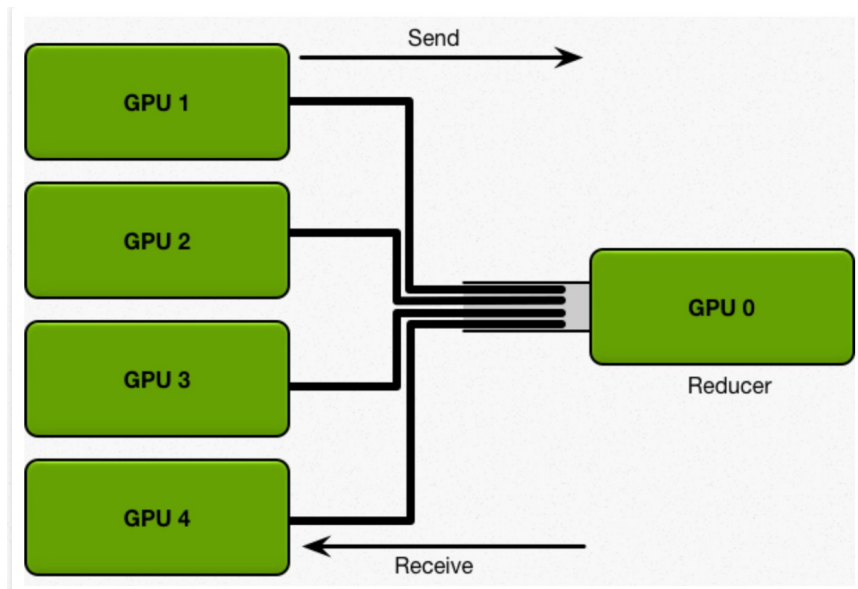
Reduction over Gradients

- To synchronize gradients of N learners, a reduction operation needs to be performed

$$\sum_{j=1}^N \Delta w_j$$

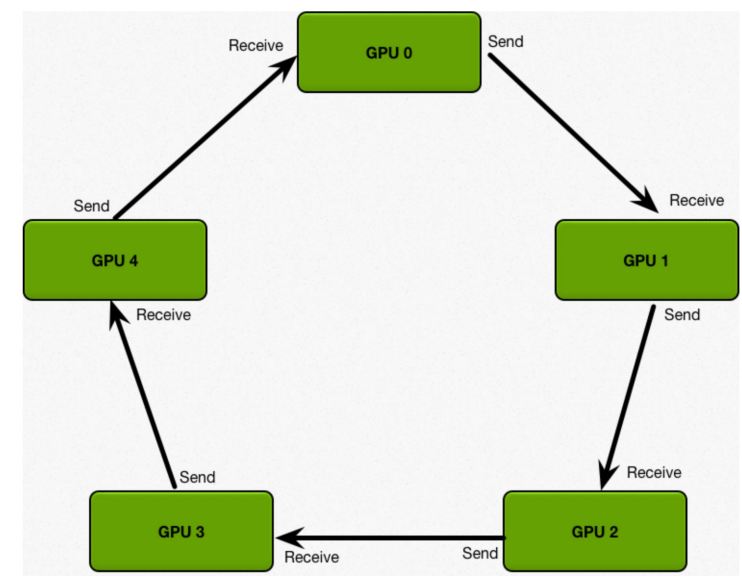
Reduction Topologies

Parameter server: single reducer



SUM (Reduce operation) performed at PS

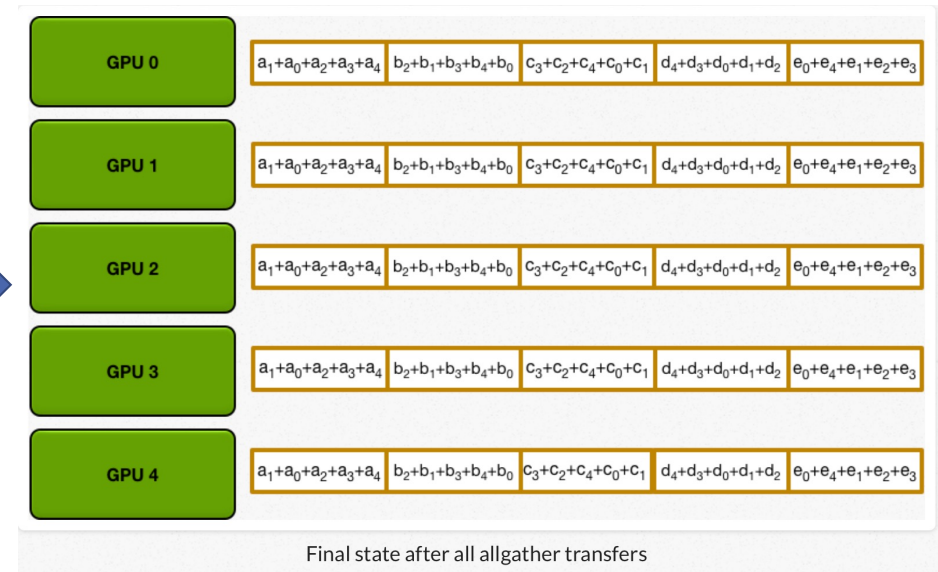
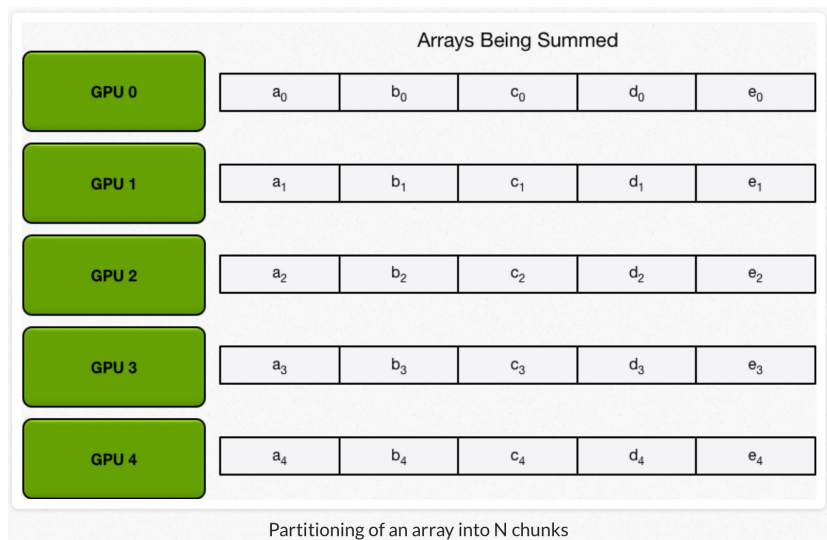
GPUs arranged in a logical Ring (aka bucket) :
all are reducers



SUM (Reduce operation) performed at all nodes

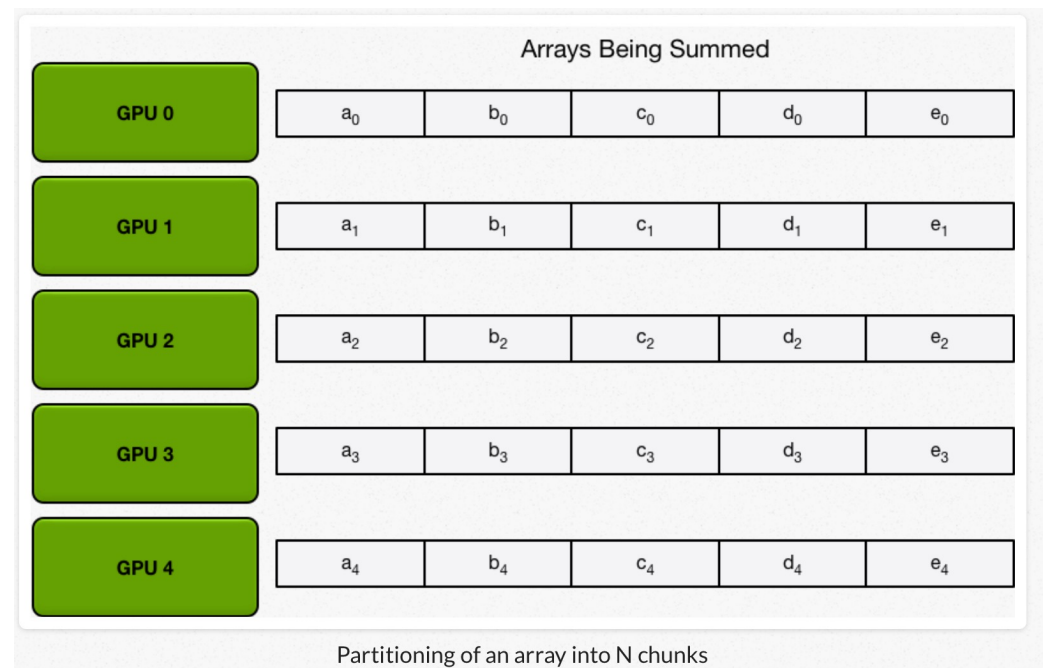
- Each node has a left neighbor and a right neighbor
- Node only sends data to its right neighbor, and only receives data from its left neighbor

Example Problem

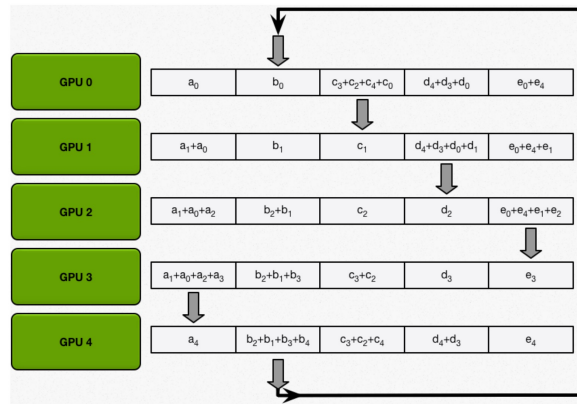
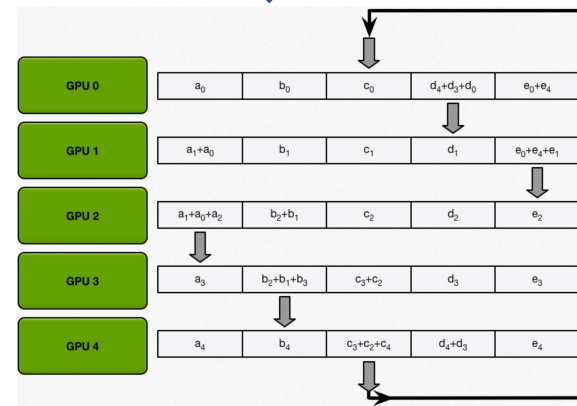
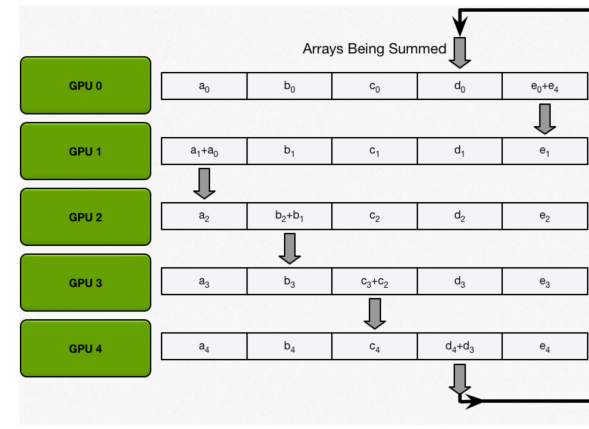
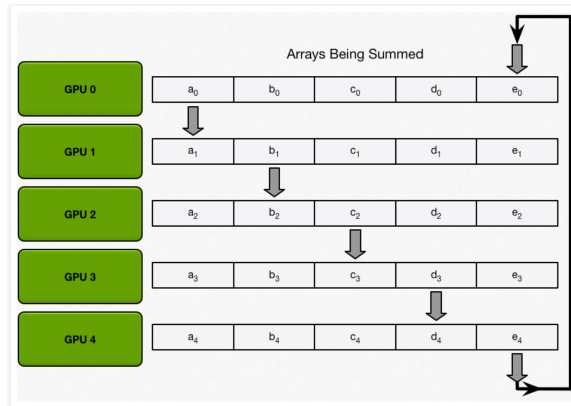
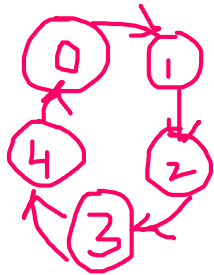


Ring All-Reduce

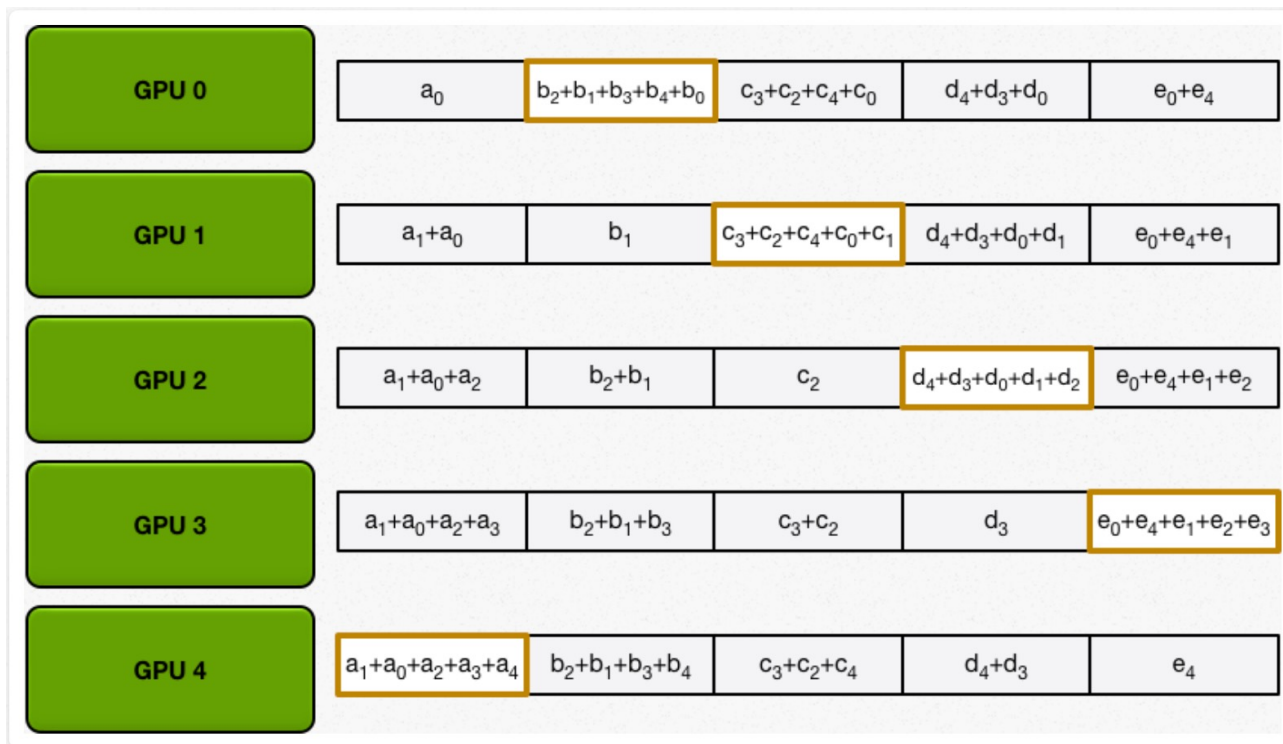
- Two step algorithm:
 - *Scatter-reduce*
 - GPUs exchange data such that every GPU ends up **with a chunk** of the final result
 - *Allgather*
 - GPUs exchange chunks from scatter-reduce such that all GPUs end up with the complete final result.



Ring All-Reduce: Scatter-Reduce Step

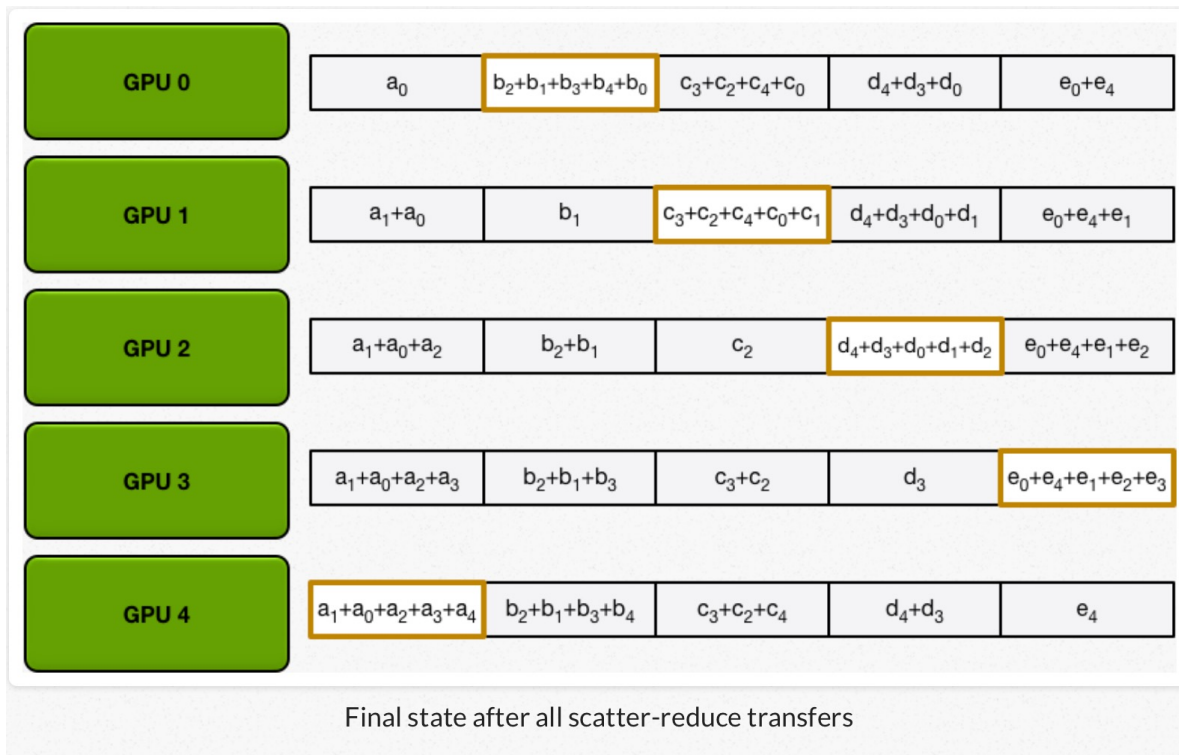


Ring All-Reduce: End of Scatter-Reduce Step



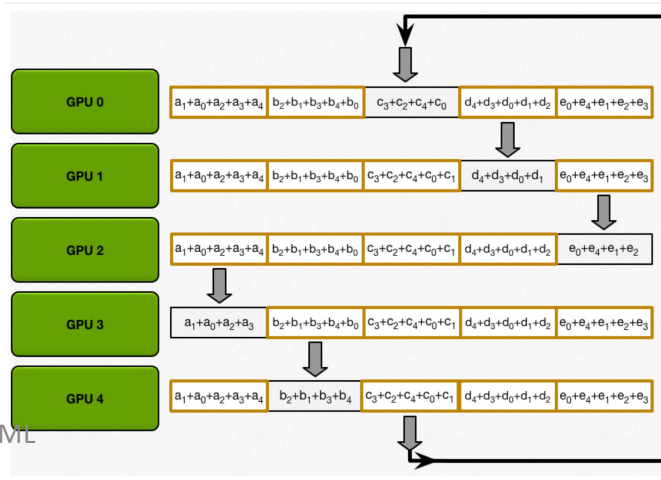
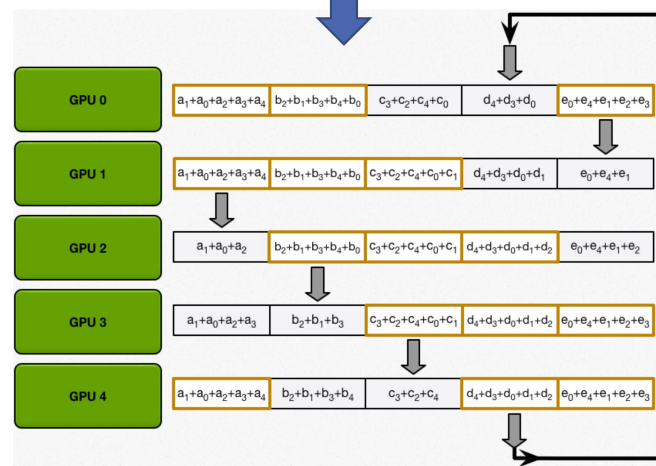
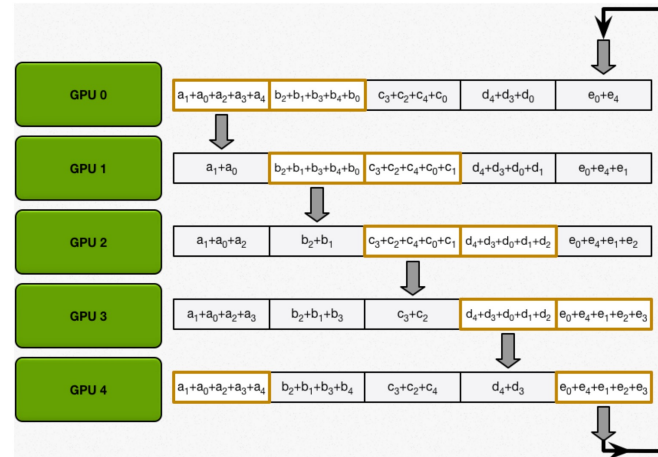
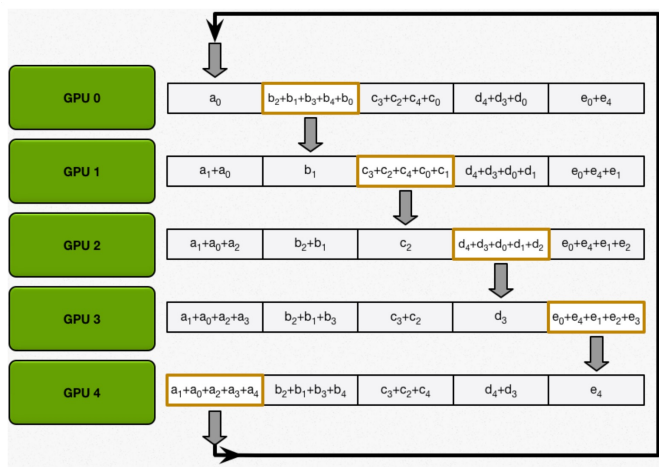
How many iterations in scatter-reduce step with N GPUs ?

Ring All-Reduce: What to do next ?

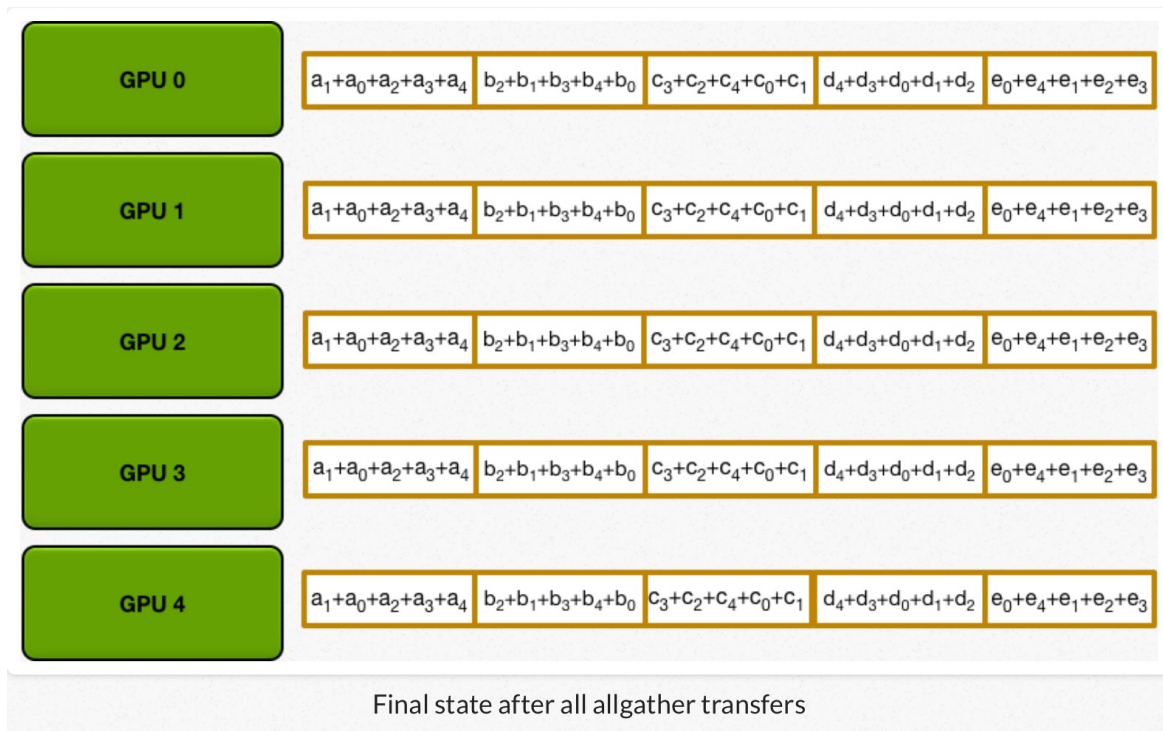


GPU	Send	Receive
0	Chunk 1	Chunk 0
1	Chunk 2	Chunk 1
2	Chunk 3	Chunk 2
3	Chunk 4	Chunk 3
4	Chunk 0	Chunk 4

Ring All-Reduce: AllGather Step



Ring All-Reduce: End of AllGather Step

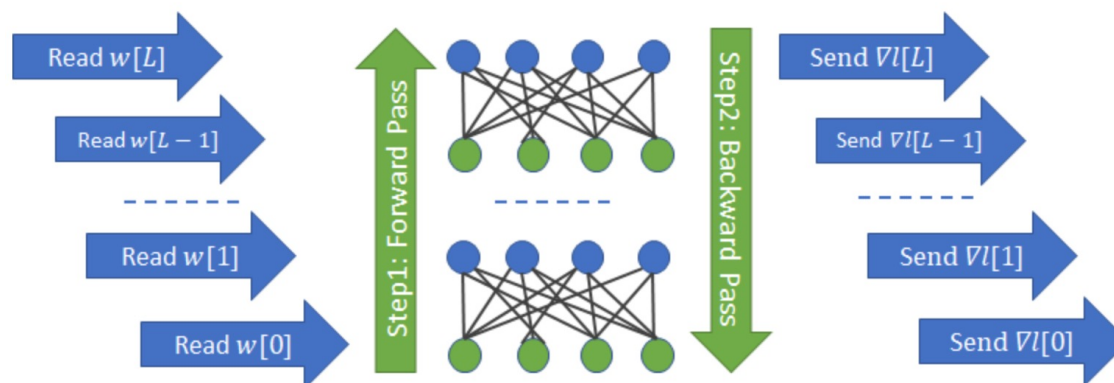


How many iterations in allgather step with N GPUs ?

Parameter Server (PS) vs Ring All-Reduce: Communication Cost

- P : number of processes N : total number of model parameters
- PS (centralized reduce)
 - Amount of data sent to PS by $(P-1)$ learner processes: $N(P-1)$
 - After reduce, PS sends back updated parameters to each learner
 - Amount of data sent by PS to learners: $N(P-1)$
 - Total communication cost at PS process is proportional to $2N(P-1)$
- Ring All-Reduce (decentralized reduce)
 - Scatter-reduce: Each process sends N/P amount of data to $(P-1)$ learners
 - Total amount sent (per process): $N(P-1)/P$
 - AllGather: Each process again sends N/P amount of data to $(P-1)$ learners
 - Total communication cost per process is $2N(P-1)/P$
- PS communication cost is proportional to P whereas ring all-reduce cost is practically independent of P for large P (ratio $(P-1)/P$ tends to 1 for large P)
- Which scheme is more bandwidth efficient ?
- Note that both PS and Ring all-reduce involve synchronous parameter updates

All-Reduce applied to Deep Learning



- Backpropagation computes gradients starting from the output layer and moving towards in the input layer
- Gradients for output layers are available earlier than inner layers
- Start all reduce on the output layer parameters while other gradients are being computed
- Overlay of communication and local compute; Pipelining

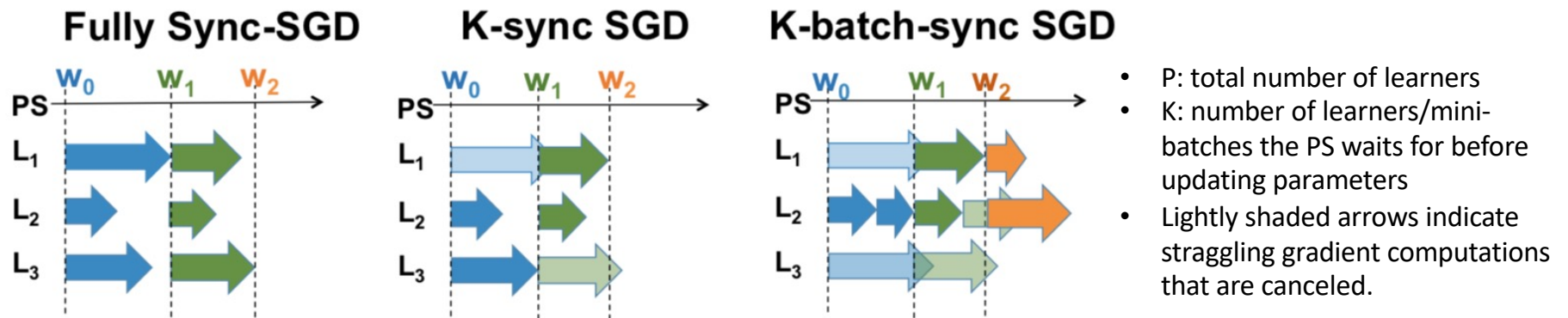
Scaling using compute-communication overlap in all-reduce

- <https://andrew.gibiansky.com/blog/machine-learning/baidu-allreduce/>

Synchronous SGD and the Straggler problem

- PS needs to wait for updated gradients from all the learners before calculating the model parameters
- Even though size of mini-batch processed by each learner is same, updates from different learners may be available at different times at the PS
 - Randomness in compute time at learners
 - Randomness in communication time between learners and PS
- Waiting for slow and straggling learners diminishes the speed-up offered by parallelizing the training

Synchronous SGD Variants



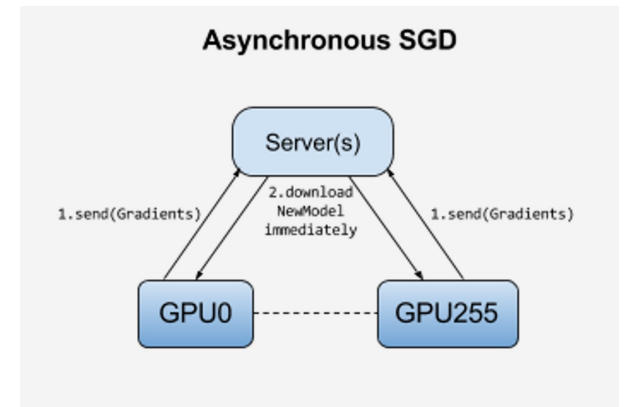
- **K-sync SGD:** PS waits for gradients from K learners before updating parameters; the remaining learners are canceled
- When $K = P$, K-sync SGD is same as Fully Sync-SGD
- **K-batch sync:** PS waits for gradients from K mini-batches before updating parameters; **the remaining (unfinished) learners are canceled**
 - Irrespective of which learner the gradients come from
 - Wherever any learner finishes, it pushes its gradient to the PS, fetches current parameter at PS and starts computing gradient on the next mini-batch based on the same local value of the parameters
- Runtime per iteration reduces with K-batch sync; error convergence is same as K-sync

Asynchronous Distributed SGD

- SGD can be made **asynchronous (ASGD)**
 - Relax timing of updates
 - Relax the synchronization restriction creating an **inconsistent model**
 - Each worker updates when it completes computing gradients
 - **Staleness (or lag)** of a model:
 - interval of time between the last update to the parameter server and the current model
 - Stragglers problem:
 - some workers may be very late (stragglers) and use a model with high staleness

Asynchronous stochastic gradient descent (ASGD)

- Each learner asynchronously repeats the following:
 - **Pull:** Get the parameters from the server
 - **Compute:** Compute the gradient with respect to randomly selected mini-batch (i.e., a certain number of samples from the dataset)
 - **Push and update:** Communicate the gradient to the server. The server then updates the parameters by subtracting this newly communicated gradient multiplied by the learning rate
- Reduces synchronization and communication overhead by tolerating stale gradient updates
- Recent analyses show ASGD converges with linear asymptotic speedup over SGD
- Examples: Downpour



DL Parallelism Approaches

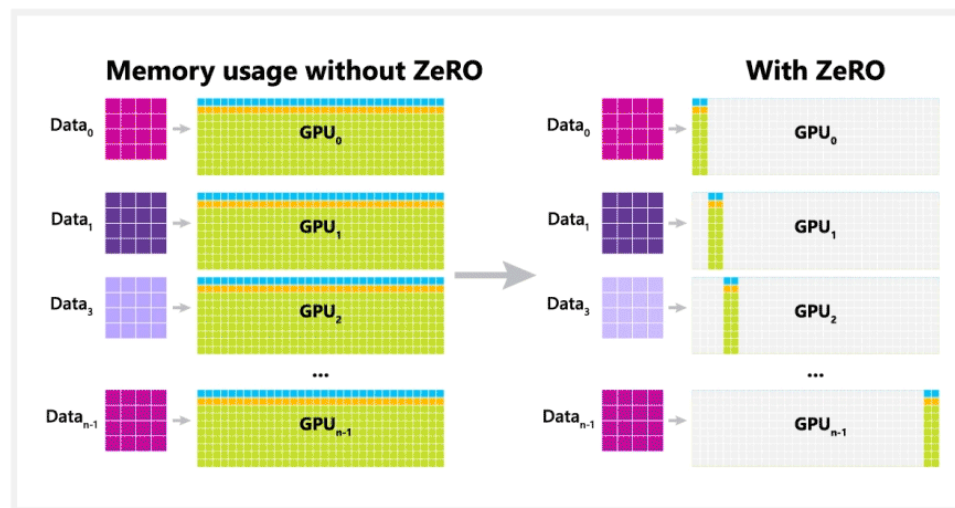
- Model Parallelism
 - Split model across multiple compute engines (GPUs)
- Pipelining
 - Use layers as pipeline to keep all compute engines busy
- Data Parallelism
 - Split data across multiple compute engines
- Hybrid Parallelism
 - Use multiple approaches together (DP + TP + PP + SP)
- **Large-Scale Training Approaches**
 - **ZeRO-1/2/3 and FSDP2 — memory-efficient sharding across GPUs**
 - **Tensor Parallelism — split weight matrices within a layer**

Motivation — Memory at Scale

- **Memory required for loading GPT-3 (175B parameters):**
 - Parameters and activations stored in fp16 (2 bytes): $175\text{B} \times 2 = 350\text{ GB}$
 - 350 GB >> 80 GB (NVIDIA H100 HBM3 capacity)
- **LLaMA 3 405B at fp16:**
 - Parameters alone: $405\text{B} \times 2\text{ bytes} = 810\text{ GB}$ — needs 10+ H100 GPUs just for weights
- **With Adam optimizer (fp32 master weights + momentum + variance):**
 - Total per-GPU memory $\approx 16N$ bytes — for 175B params: $\sim 2.8\text{ TB}$ on a single GPU
- **Solution: ZeRO (Zero Redundancy Optimizer) + FSDP2**

DeepSpeed + ZeRO: Memory Usage Overview

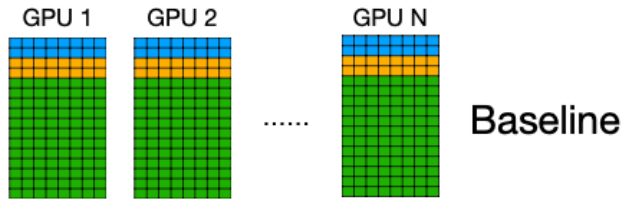
DeepSpeed + ZeRO



Source: Rajbhandari et al. (2020). ZeRO & DeepSpeed. Microsoft Research Blog. microsoft.com/en-us/research/blog/zero-deepspeed-new-system-optimizations-enable-training-models-with-over-100-billion-parameters/ — Left: each GPU stores full model (redundant). Right: With ZeRO, each GPU stores only 1/N of model state.

DeepSpeed Zero

DeepSpeed Zero to Optimize Memory



■: weights, 2 bytes

■: gradients, 2 bytes

■: optimizer states, K bytes for Adam

* K=12 from ZeRO: “fp32 copy of the parameters, momentum and variance”

N : number of GPUs ψ : number of parameters

- Naive data parallelism has N copies of weights, gradients and optimizer states.
- This introduces redundancy and easily leads to OOM when training large models.

$$\left(\underbrace{2\text{bytes}}_{\text{weights}} + \underbrace{2\text{bytes}}_{\text{gradients}} + \underbrace{12\text{bytes}}_{\text{optim states}} \right) \psi$$

- Even for most most advanced A100/H100 with 80GB memory, the largest trainable model is

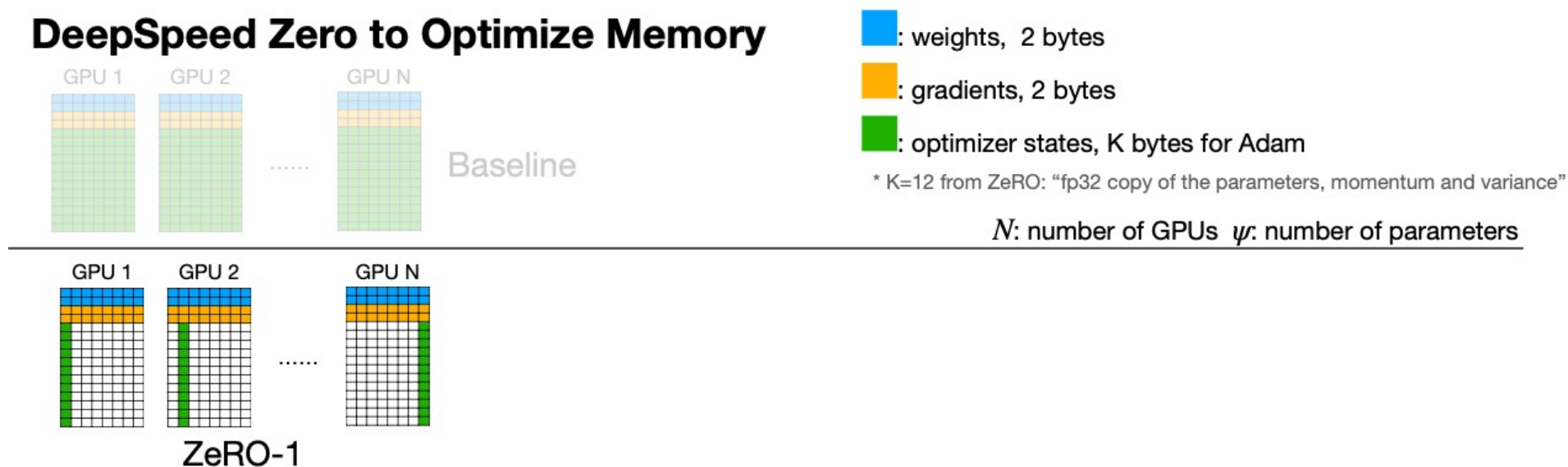
$$80\text{GB}/16\text{Bytes} = 5.0B$$

which is far away from current SOTA (175B).

Slide courtesy of song Han

DeepSpeed Zero-1

DeepSpeed Zero to Optimize Memory



- Zero-1 proposes to partition / shard **optimizer states**.

Memory Consumption

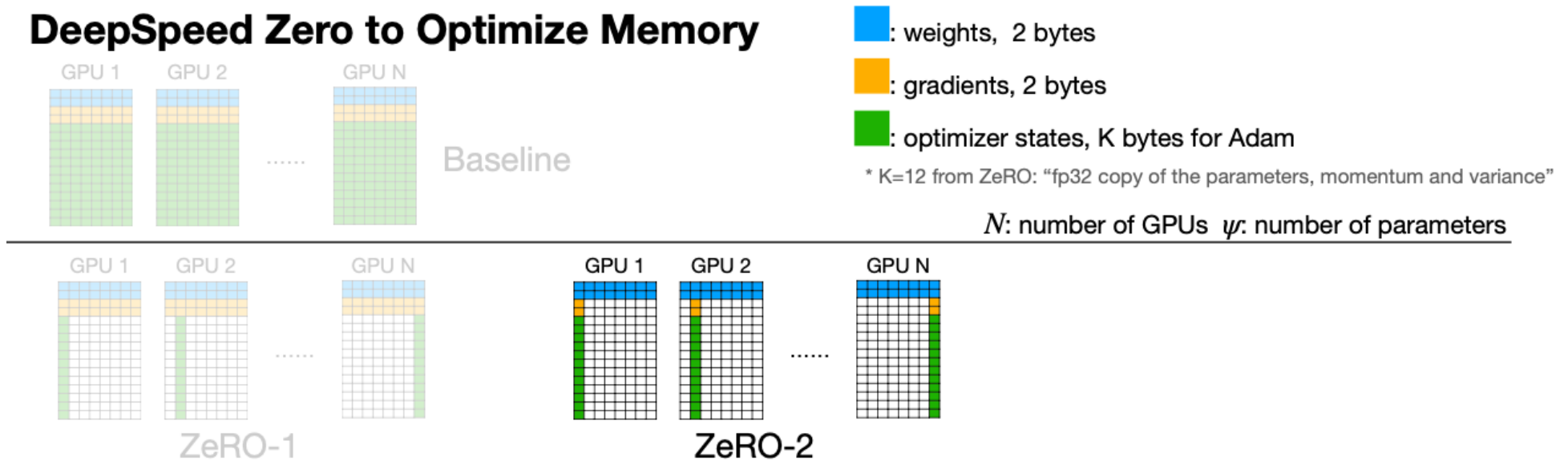
$$\left(\underbrace{2\text{bytes}}_{\text{weights}} + \underbrace{2\text{bytes}}_{\text{gradients}} + \underbrace{12/N\text{bytes}}_{\text{optim states}} \right) \psi$$

Largest Model Size for Training ($N=64$)

$$80\text{GB}/4.2\text{Bytes} = 19B$$

DeepSpeed Zero-2

DeepSpeed Zero to Optimize Memory



- Zero-2 proposes to partition / shard **optimizer states** and **gradients**.

Memory Consumption

$$(\underbrace{2\text{bytes}}_{\text{weights}} + \underbrace{2/N\text{bytes}}_{\text{gradients}} + \underbrace{12/N\text{bytes}}_{\text{optim states}})\psi$$

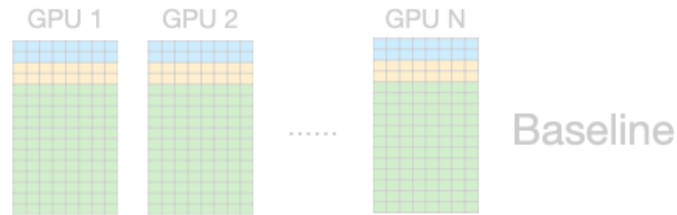
Largest Model Size for Training ($N=64$)

$$80\text{GB}/2.2\text{Bytes} = 36B$$

Slide courtesy of song Han

DeepSpeed Zero-3

DeepSpeed Zero to Optimize Memory



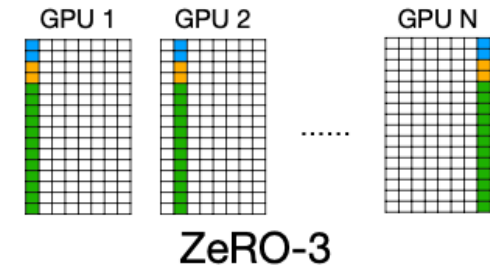
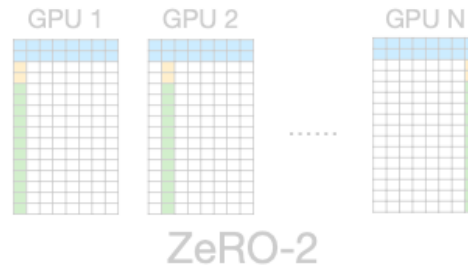
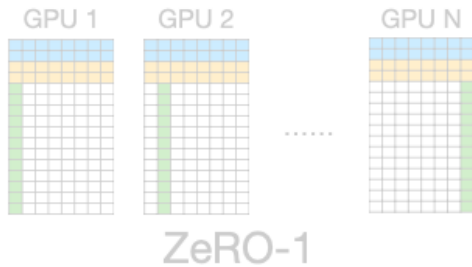
■: weights, 2 bytes

■: gradients, 2 bytes

■: optimizer states, K bytes for Adam

* K=12 from ZeRO: "fp32 copy of the parameters, momentum and variance"

N : number of GPUs ψ : number of parameters



- Zero-3 proposes to partition / shard **optimizer states, gradients and weights**.

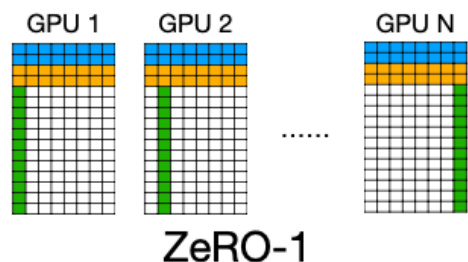
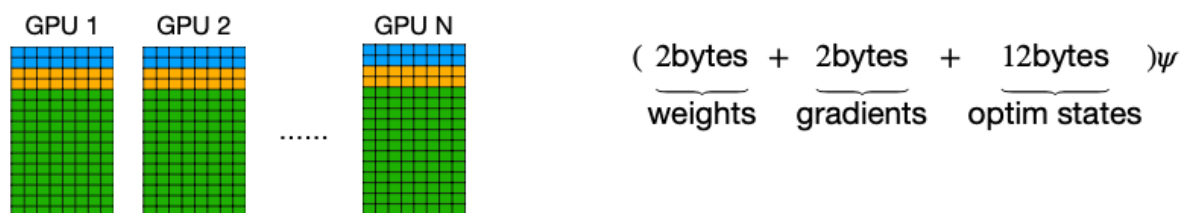
Memory Consumption

$$\underbrace{(2/N\text{bytes})}_{\text{weights}} + \underbrace{(2/N\text{bytes})}_{\text{gradients}} + \underbrace{(12/N\text{bytes})}_{\text{optim states}} \psi$$

Largest Model Size for Training ($N=64$)

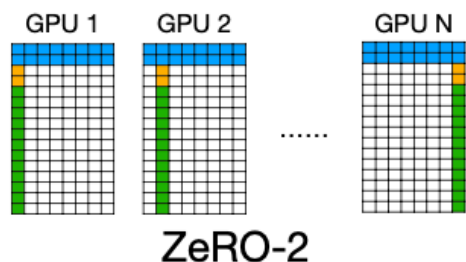
$$80\text{GB}/0.25\text{Bytes} = 320B$$

DeepSpeed Zero-1/2/3 and FSDP



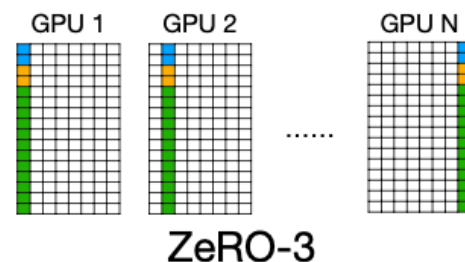
$$\left(\underbrace{2\text{bytes}}_{\text{weights}} + \underbrace{2\text{bytes}}_{\text{gradients}} + \underbrace{12/N\text{bytes}}_{\text{optim states}} \right) \psi$$

$$80\text{GB}/4.2\text{Bytes} = 19B$$



$$\left(\underbrace{2\text{bytes}}_{\text{weights}} + \underbrace{2/N\text{bytes}}_{\text{gradients}} + \underbrace{12/N\text{bytes}}_{\text{optim states}} \right) \psi$$

$$80\text{GB}/2.2\text{Bytes} = 36B$$



$$\left(\underbrace{2/N\text{bytes}}_{\text{weights}} + \underbrace{2/N\text{bytes}}_{\text{gradients}} + \underbrace{12/N\text{bytes}}_{\text{optim states}} \right) \psi$$

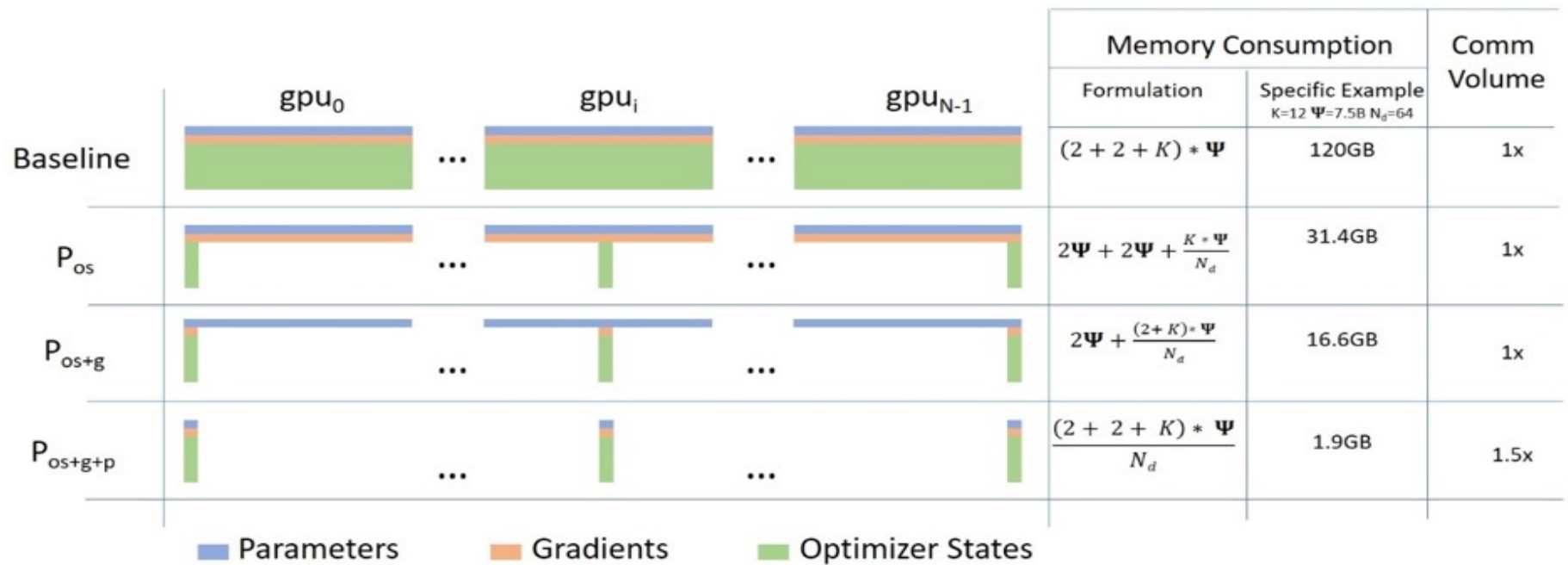
$$80\text{GB}/0.25\text{Bytes} = 320B$$

In PyTorch, ZeRO-3 is implemented via `FullyShardedDataParallel`, known as FSDP.

DeepSpeed ZeRO — Memory Optimization Stages

- **ZeRO-1: Shard Optimizer States**
 - Optimizer states (fp32 params, momentum, variance) sharded across GPUs
 - Each GPU stores $1/N$ of optimizer states $\rightarrow \sim 4\times$ memory reduction
- **ZeRO-2: + Shard Gradients**
 - Gradient sharding added on top of ZeRO-1
 - Each GPU computes and stores $1/N$ of gradients $\rightarrow \sim 8\times$ memory reduction
- **ZeRO-3: + Shard Parameters**
 - All parameters also sharded — parameters all-gathered on demand
 - Maximum memory savings: $\sim 64\times$ with $N=64$ GPUs
- ZeRO is data parallelism with sharding — no change to forward/backward semantics

ZeRO Stages: Memory Breakdown per GPU



Source: Rajbhandari et al. (2020). ZeRO & DeepSpeed. Microsoft Research Blog. microsoft.com/en-us/research/blog/zero-deepspeed-new-system-optimizations-enable-training-models-with-over-100-billion-parameters/ — P_{os}: optimizer state sharding. P_{os+g}: + gradient sharding. P_{os+g+p}: + parameter sharding (ZeRO-3). For Ψ=7.5B params, K=12 (Adam), N=64 GPUs.

ZeRO & DeepSpeed: Scale, Speed, Cost, Usability

Scale

- 100B parameter
- 10X bigger

Speed

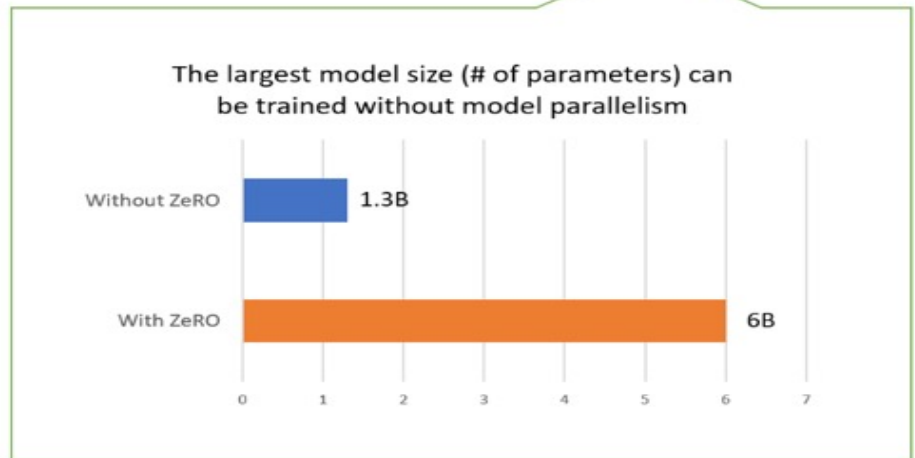
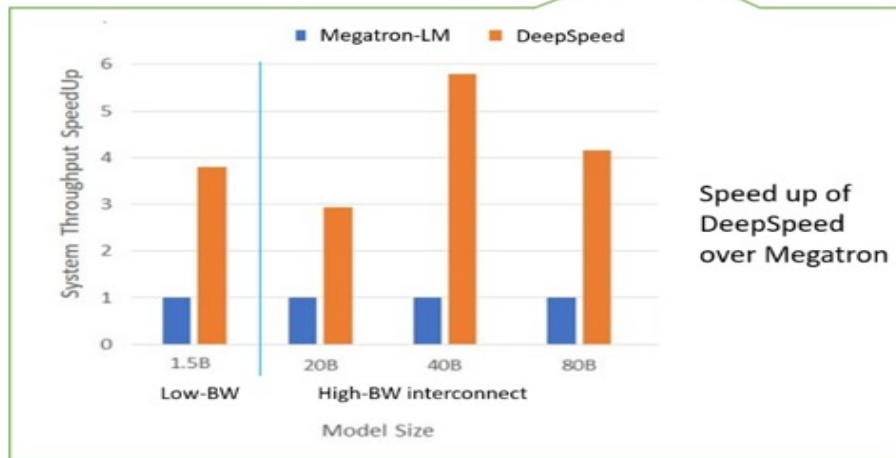
- Up to 5X faster

Cost

- Up to 5X cheaper

Usability

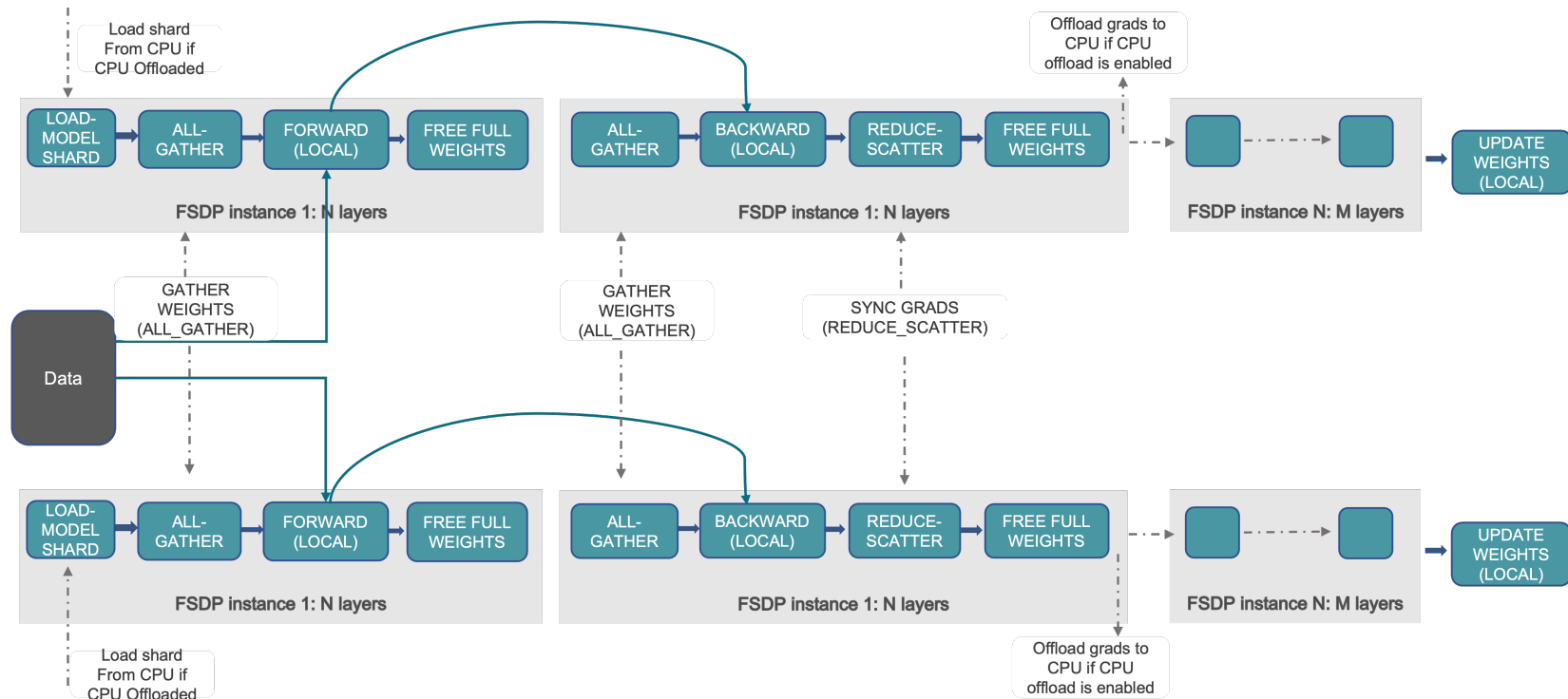
- Minimal code change



Source: Rajbhandari et al. (2020). ZeRO & DeepSpeed. Microsoft Research Blog. microsoft.com/en-us/research/blog/zero-deepspeed-new-system-optimizations-enable-training-models-with-over-100-billion-parameters/ — Left chart: DeepSpeed up to 5× faster than Megatron-LM across model sizes. Right: ZeRO enables 6B params on single GPU vs 1.3B without ZeRO. HPML

FSDP in PyTorch

- https://pytorch.org/tutorials/intermediate/FSDP_tutorial.html



FSDP in PyTorch — FSDP2

FSDP2 is now the recommended default in PyTorch 2.4+. API changed: use `fully_shard()` not `FullyShardedDataParallel()`.

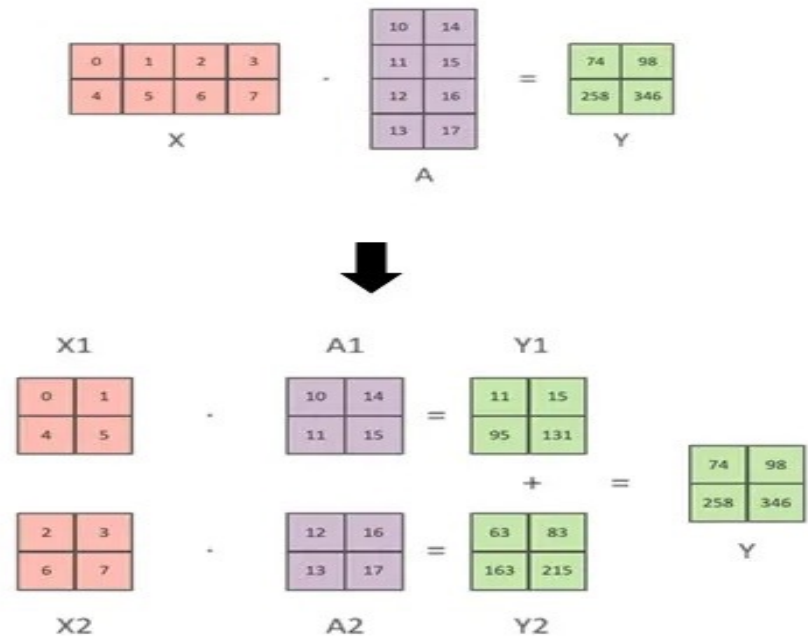
	FSDP1	FSDP2 (current — 2024+)
API class	<code>FullyShardedDataParallel()</code>	<code>fully_shard()</code> — functional API
Sharding repr.	Flat-parameter groups	Per-parameter DTensor (dim-0)
Memory mgmt	<code>record_stream()</code> — non-deterministic	Deterministic release, ~7% less GPU RAM
State dict	Requires all-gather comms	Communication-free DTensor sharding
Float8 support	Not supported	✓ 50% throughput gain (IBM+Meta, 2024)

```
from torch.distributed.fsdp import fully_shard
model = Transformer()
for layer in model.layers: fully_shard(layer)
fully_shard(model)
optim = torch.optim.Adam(model.parameters(), lr=1e-3)
```

Tensor Parallelism & 4D Parallelism

Tensor Parallelism: Matrix Partitioning

- **Key idea: $X \cdot A = Y$**
 - Split matrix A into sub-matrices A1, A2
 - $X1 \cdot A1$ and $X2 \cdot A2$ computed in parallel on separate GPUs
 - Partial results combined: $Y1 + Y2 = Y$
- **Why this matters:**
 - Each GPU only needs to store half the weight matrix
 - Computation parallelized within the layer itself
 - Communication only at layer output (AllReduce)



Source: Hugging Face Model Parallelism Guide. Via Matsuzaki (2023) tsmatz.wordpress.com

Tensor Parallelism

(torch.distributed.tensor.parallel)

- Split individual weight matrices across GPUs within a node
- Native PyTorch API since PyTorch 2.3
- **Column-parallel linear:**
 - W split along output dim → AllReduce output
- **Row-parallel linear:**
 - W split along input dim → Reduce-Scatter
- **Sequence Parallel:**
 - LayerNorm/Dropout sharded on sequence dim — saves activation memory
- Use TP within a node (NVLink fast) and FSDP2 across nodes

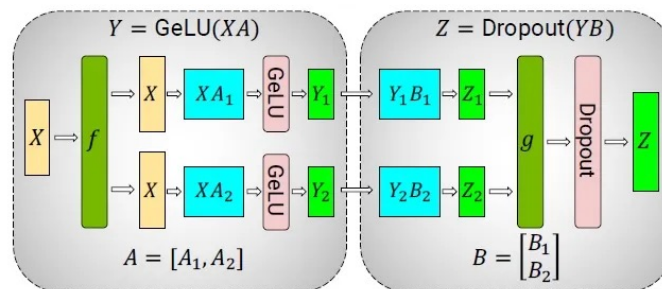
```
from torch.distributed.device_mesh import (
    init_device_mesh)
from torch.distributed.tensor.parallel import (
    parallelize_module, ColwiseParallel)

# dp=4 nodes, tp=8 GPUs per node
mesh = init_device_mesh("cuda", (4, 8),
    mesh_dim_names=("dp", "tp"))

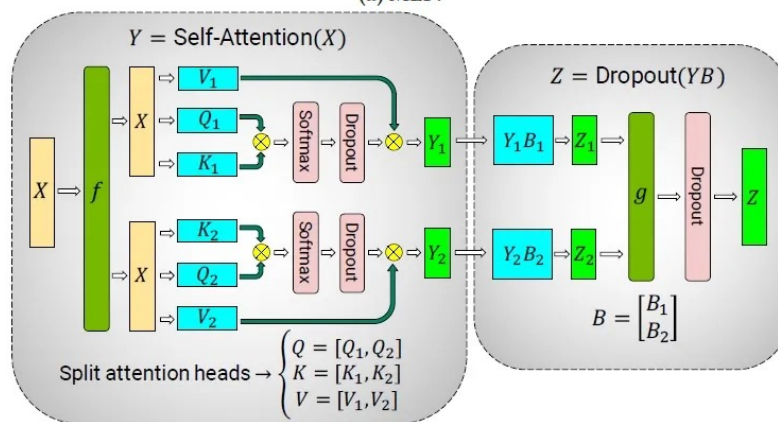
parallelize_module(
    model,
    mesh["tp"],
    ColwiseParallel()) # apply TP

# Then wrap with FSDP2 on dp dim
fully_shard(model, mesh=mesh["dp"])
```

Megatron-LM Tensor Parallelism: MLP and Attention



(a) MLP.



(b) Self-Attention.

Source: Narayanan et al. (2021). Megatron-LM. SC'21. Via Matsuzaki (2023) tsmatz.wordpress.com via Matsuzaki, T. (2023). Implement Model Parallelism in LLMs. tsmatz.wordpress.com/2023/09/21/model-parallelism/ — (a) MLP block: column-parallel A followed by row-parallel B; one AllReduce. (b) Self-Attention: Q,K,V heads split across GPUs; one AllReduce at projection output.

4D Parallelism — TorchTitan (2024–2025)

- **Combining four parallelism dimensions for extreme-scale LLM training:**
- **Data Parallel (DP):**
 - Outermost dimension — replicate across nodes using FSDP2 or DDP
- **Tensor Parallel (TP):**
 - Split weight matrices within a node — 8-GPU TP on NVLink (900 GB/s)
- **Pipeline Parallel (PP):**
 - Layer groups across node groups — zero-bubble scheduling (ZB-H1)
- **Sequence Parallel (SP):**
 - LayerNorm/Dropout sharded on sequence dim — enables 1M+ token context
- **TorchTitan results: LLaMA 3.1 405B at 512 GPUs → +30% over optimized baseline**

Other Techniques: MoE (Mixture of Experts)

Parallelism

- MoE introduces multiple expert networks within a larger model architecture
- During training/inference, only a subset of experts (e.g., 2 out of 64) activated per input
- Creates conditional computation — different inputs activate different sub-models
- **Benefits:**
 - Scale model capacity massively without linear increase in compute
 - Lower FLOPs per token vs dense models of similar capacity
 - Adaptable to diverse inputs and tasks
- **Requires:**
 - Sparse routing mechanisms (e.g., top-k routing from gating network)
 - Balancing mechanisms to prevent expert collapse (load imbalance)
- Used in: Mixtral 8x7B, DeepSeek-MoE, Switch Transformer, GShard

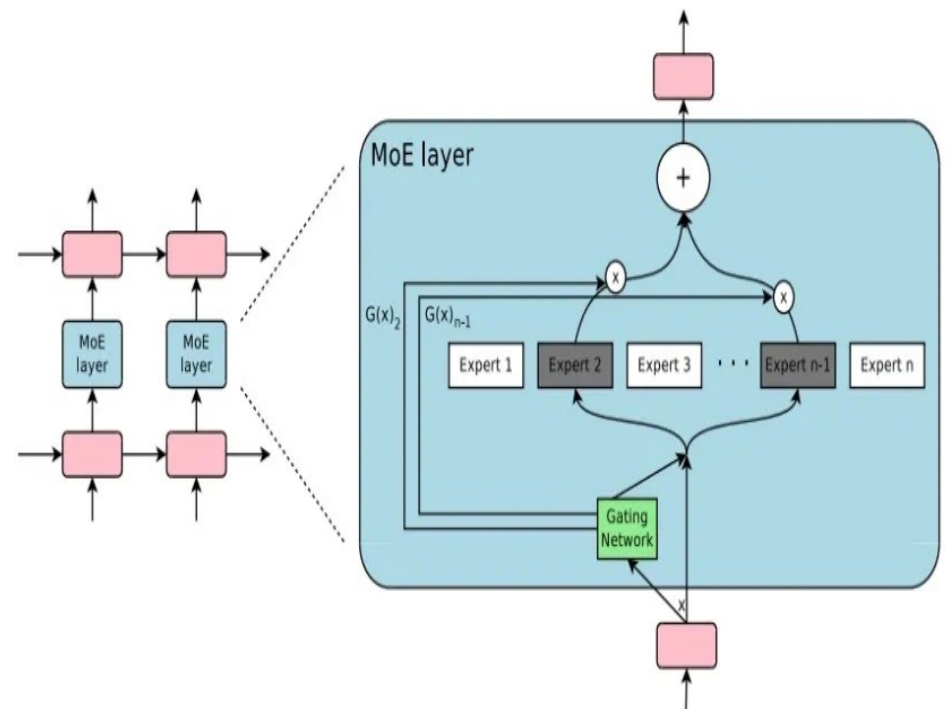
Sparsely-Gated Mixture of Experts (MoE)

- **Left: MoE layer stack in a transformer**

- MoE layers replace standard FFN/MLP blocks
- Stacked across transformer depth

- **Right: Single MoE layer detail**

- Gating network selects top-k experts (e.g. 2 of n)
- $G(x)$ = gating weights, x = token input
- Only activated experts perform computation
- Outputs weighted and summed



Source: Shazeer et al. (2017). Sparsely-Gated MoE. ICLR. Via Matsuzaki (2023) tsmatz.wordpress.com

Distributed Deep Learning Strategies — Full Taxonomy

Data Parallelism	Model / Pipeline Parallelism	Memory & Sharding	SGD & Sync Strategies	Advanced & Hybrid
DDP DistributedDataParallel Ring AllReduce	Model Parallelism Layer-level split across GPUs	ZeRO-1 Shard optimizer states only	Sync SGD All P workers deterministic	3D Parallelism DP + TP + PP Megatron-LM
DataParallel Single machine legacy (GIL limit)	Pipeline (GPipe) Microbatch inject O(M) memory	ZeRO-2 ZeRO-1 + Gradient shard	K-sync SGD Wait K < P cancel stragglers	4D Parallelism DP+TP+PP+SP TorchTitan
Horovod MPI/NCCL backend ring AllReduce	Interleaved 1F1B Backward-first memory efficient	ZeRO-3 ZeRO-2 + Parameter shard	K-batch Sync Wait K mini-batches, any rank	Sequence Parallel LayerNorm/Dropout sharded on seq dim
Parameter Server Centralized PS (legacy, deprecated)	Zero-Bubble PP ZB-H1/H2 — defer weight grad	FSDP1 PyTorch ZeRO-3 (legacy API)	Async SGD No wait; gradient staleness risk	MoE / Expert Par. Sparse gating per-token routing
ASGD Async SGD Downpour-style	Tensor Parallel (TP) Column/Row-parallel within a layer	FSDP2 fully_shard() DTensor-based	Elastic Training torchrun; auto-remove failed nodes	DeviceMesh / TP N-D communicator PyTorch 2.3+

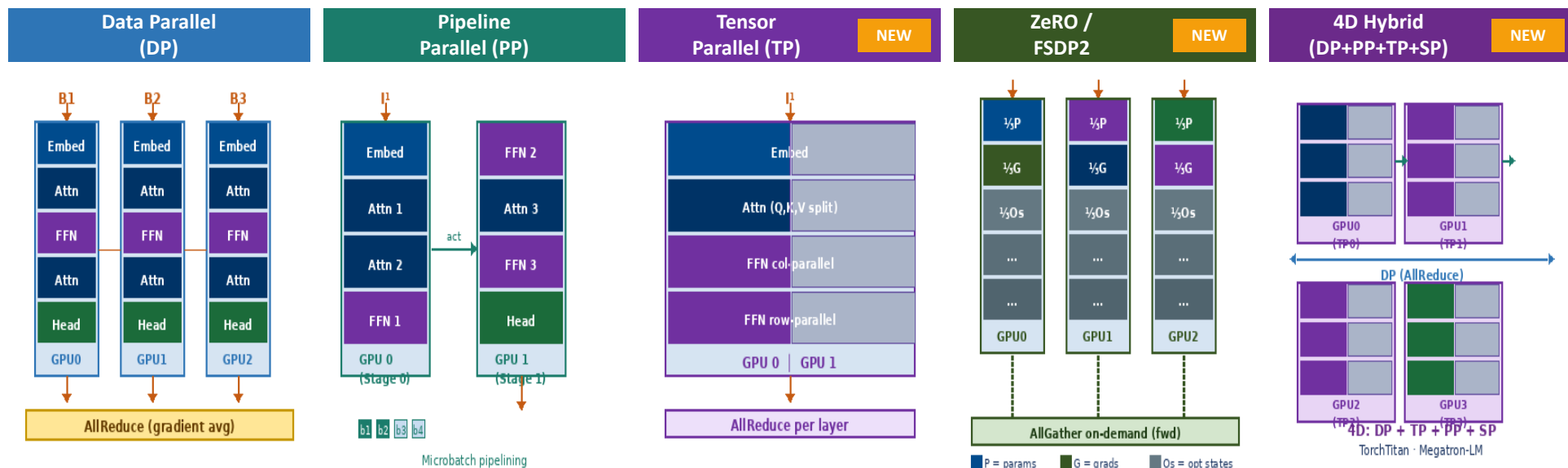
Legend:
■ Data Parallelism family
■ Model/Pipeline Parallelism family
■ Memory & Sharding family
■ SGD / Optimization family
■ Advanced & Hybrid
■ Legacy / deprecated

Key LLMs and Their Distributed Training Strategies

Model	Params	Data Par. (DP)	Tensor Par. (TP)	Pipeline Par. (PP)	ZeRO / FSDP2	MoE / Expert	Seq. Parallel	Framework / Infrastructure
GPT-3	175B	✓	✓	✓	🕒	—	—	Megatron-style · Azure 10K GPU cluster
GPT-4	~1.8T*	✓	✓	✓	✓	✓*	—	DeepSpeed + Megatron · Azure cluster
LLaMA 3 405B	405B	✓	✓	✓	✓	—	🕒	FSDP2 · PyTorch · 16K H100 cluster
Gemini Ultra	~1T*	✓	✓	✓	—	✓*	✓	JAX/XLA · TPU v4/v5 pod · custom infra
Megatron-LM 530B	530B	✓	✓	✓	—	—	🕒	Megatron-LM 3D · 4480 A100 (Azure)
Mixtral 8×22B	141B (39B act)	✓	✓	✓	🕒	✓	—	DeepSpeed / vLLM inference · MoE expert par.
DeepSeek-V3	671B (37B act)	✓	✓	✓	✓	✓	✓	DualPipe PP · MoE · FP8 mixed-precision
BLOOM 176B	176B	✓	✓	✓	✓	—	—	Megatron-DeepSpeed · 384 A100 (Jean Zay)
Granite 3.x	2–8B	✓	—	—	✓	—	—	FSDP2 · TorchTitan · IBM Research cluster
Switch Trans.	1.6T	✓	—	—	—	✓	—	TPU v3 · Mesh TensorFlow · expert par.

DL Parallelism Strategies for

Five primary strategies for scaling model training across multiple GPUs and nodes



Data: Different mini-batches per GPU

Model: Full model replica on each GPU

Comm: AllReduce grads after backward

Memory: Full model $\times$ N GPUs (redundant)

Use when: Model fits in one GPU. Most common.

Data: Micro-batches pipelined in sequence

Model: Layer groups split across GPUs

Comm: P2P activations at stage boundaries

Memory: Full model sharded by stage

Use when: Model too deep for one GPU; need throughput.

Data: Single input broadcast to all GPUs

Model: Weight matrices split within each layer

Comm: AllReduce per layer (NVLink required)

Memory: Each GPU holds $1/N$ of weight matrices

Use when: Individual layers too large for one GPU.

Data: Batches across GPUs (like DP)

Model: Params + grads + opt-states all sharded

Comm: AllGather (fwd) + Reduce-Scatter (bwd)

Memory: Up to $64\times$ reduction (ZeRO-3)

Use when: Optimizer states overflow GPU memory.

Data: Batches across DP node groups

Model: TP within node $\cdot$ PP across nodes $\cdot$ SP

Comm: AllReduce(TP) + P2P(PP) + AllReduce(DP)

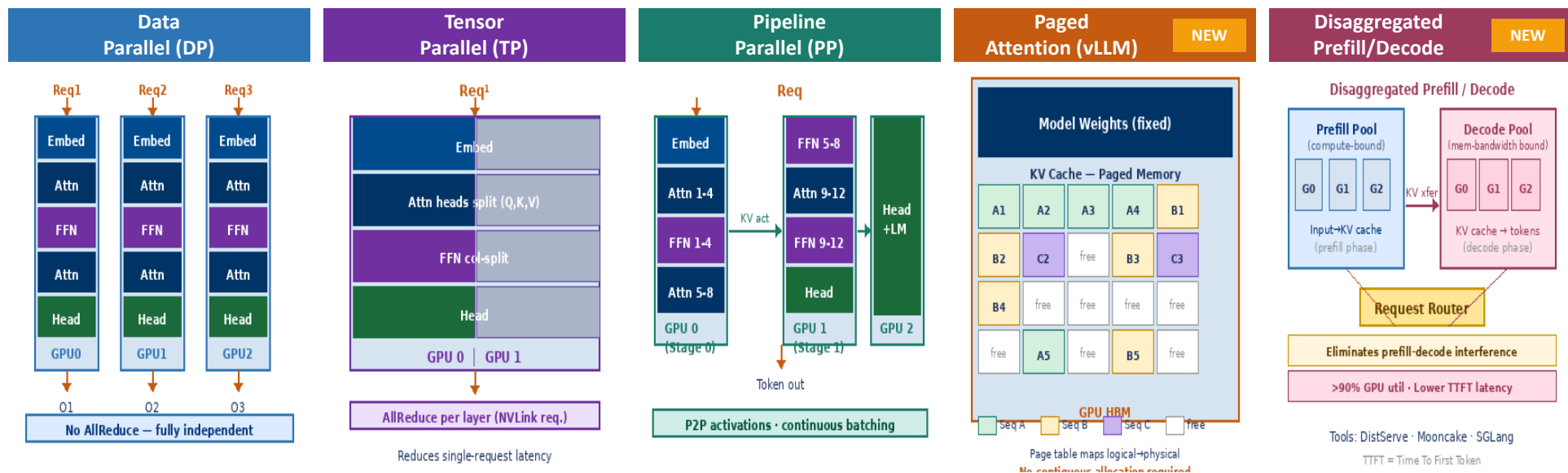
Memory: Minimal per-GPU via combined sharding

Use when: 100B+ params at cluster scale.

¹ HuggingFace Transformers — Model Parallelism Guide (huggingface.co/docs/transformers/perf_train_gpu_many)
HPML

DL Parallelism Strategies for

Inference is memory-bandwidth bound — no backward pass, no AllReduce for gradients; focus on latency, throughput, and KV cache



Data: 1 request per GPU replica

Model: Full model replica on each GPU

Comm: None — requests fully independent

Perf: Low latency; scales with replica count

Use when: Many concurrent users; latency SLA.

Data: Single request broadcast to all GPUs

Model: Attention heads + FFN split across GPUs

Comm: AllReduce per layer (NVLink only)

Perf: Reduces single-request decode latency

Use when: Lowest latency for one request at a time.

Data: 1 request, stage-by-stage

Model: Layers split into pipeline stages

Comm: P2P: KV activations between stages

Perf: Good throughput; continuous batching fills it

Use when: Very deep models; prefill/decode separated.

Data: Many concurrent requests, variable lengths

Model: Full model; KV cache in paged virtual memory

Comm: None — single GPU (or TP for large models)

Perf: 2–4× more requests; eliminates fragmentation

Use when: Production serving of long-context LLMs.

Data: Prefill and decode requests routed separately

Model: Separate GPU pools for each phase

Comm: KV cache transferred from prefill→decode pool

Perf: >90% GPU util; eliminates prefill-decode interference

Use when: High-throughput serving; TTFT optimization.

¹ Kwon et al. (2023) vLLM · Yu et al. (2022) Orca · Zhong et al. (2024) DistServe · Hugging Face Inference Docs
HPML

Distributed Deep Learning Benchmarking Methodology

- **Key metrics:**
 - Speedup: training time on 1 GPU / time on N GPUs
 - Scaling efficiency: fraction of linear speedup achieved
 - Accuracy and end-to-end training time to target accuracy
 - Communication overhead: fraction of time in AllReduce / AllGather
- **Meaningful comparison requires:**
 - Same neural network (ResNet-50/ImageNet or LLM + C4 are standard)
 - Same hardware tier (H100 NVL vs A100 vs V100)
 - Same batch size per GPU — not global batch size
 - Train to same target accuracy — not just same number of epochs

LLM Inference at Scale

LLM Inference at Scale — A Critical New Dimension

- **Inference is now as important as training:**
 - Production LLM serving handles millions of requests/day
 - Strict SLA constraints: latency, throughput, cost per token
 - Inference bottleneck: memory-bandwidth bound (not compute-bound like training)
- **Paged Attention (vLLM):**
 - KV cache stored in non-contiguous pages — like OS virtual memory
 - Eliminates memory fragmentation → 2–4× more requests per GPU
- **Continuous Batching:**
 - Requests join/leave batch after each token — no full-batch waiting
 - GPU utilization increases from ~30% to >90%
- **KV Cache Sharding / Disaggregated Serving:**
 - KV cache sharded via tensor-parallel attention for very long contexts

PyTorch Distributed DL

Data Parallelism: Multiple GPU Configurations

- PyTorch provides several options for data-parallel training:
1. **Single-device training:**
 - Data and model fit in one GPU — no distribution needed
 - 2.
 3. **DataParallel (single machine, multi-GPU):**
 - Minimal code changes but limited by GIL and load imbalance
 - 4.
 5. **DistributedDataParallel (single machine, multi-GPU):**
 - Multi-process, faster than DataParallel — recommended for production
 - 6.
 7. **Multi-machine DDP + torchrun:**
 - Scales across node boundaries — the production standard
 - 8.
 9. **FSDP2:**
 - When model doesn't fit in GPU memory — use with DDP on top

DataParallel Example

- **torch.nn.DataParallel:**
 - Replicates model on each GPU
 - Divides each batch into N parts (N devices)
 - Backward: gradients summed into original module
- **Limitations:**
 - Single-process multi-thread → GIL contention
 - All gradients gathered on output_device — bottleneck
 - Very easy to use but usually not best performance
 - Use DDP instead for production workloads

```
class DataParallelModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.block1 = nn.Linear(10, 20)
        # wrap block2 in DataParallel
        self.block2 = nn.Linear(20, 20)
        self.block2 = nn.DataParallel(
            self.block2)
        self.block3 = nn.Linear(20, 20)

    def forward(self, x):
        x = self.block1(x)
        # Parallel on GPUs:
        x = self.block2(x)
        x = self.block3(x)
        return x

model = nn.DataParallel(model)
model.to(device)
```


torch.nn.DistributedDataParallel (DDP)

- Multi-process parallelism — one process per GPU
- Splitting input across devices in batch dimension
- Module replicated on each machine and device
- Gradients from each node averaged during backward
- **DDP is significantly faster than DataParallel:**
 - No GIL contention — multi-process not multi-thread
 - Each GPU runs its own optimizer step
 - Works natively across multiple machines

```
import torch.distributed as dist
from torch.nn.parallel import (
    DistributedDataParallel as DDP)

dist.init_process_group(backend='nccl')
rank = dist.get_rank()
device = torch.device(f'cuda:{rank}')

model = MyModel().to(device)
model = DDP(model, device_ids=[rank])

for data, target in dataloader:
    data = data.to(device)
    target = target.to(device)
    optimizer.zero_grad()
    output = model(data)
    loss = criterion(output, target)
    loss.backward() # AllReduce here
    optimizer.step()
```

Distributed Data Parallel in Multi-host Setting

- **Steps in multi-host setting:**
 - Launched on each host independently — one process per GPU
 - `init_process_group` synchronizes processes via shared file or TCP endpoint
 - Each process loads non-overlapping data via `DistributedDataSampler`
- **`torch.distributed` backends:**
 - NCCL — use for distributed GPU training (recommended)
 - Gloo — use for distributed CPU training or debugging
 - MPI — available if PyTorch built from source with MPI support
- `DistributedDataSampler` ensures each GPU sees a unique non-overlapping subset

PyTorch – Launching Distributed Training

⚠ *mpirun is no longer recommended. torch.distributed.launch is deprecated. Use torchrun instead.*

- **torchrun — the current standard launcher:**
 - Fault-tolerant: auto-restarts crashed workers up to max_restarts
 - Elastic: nodes can join or leave; world_size auto-adjusts
 - Sets RANK, LOCAL_RANK, WORLD_SIZE automatically
 - SLURM-native: works with srun without mpirun wrapper

```
Old (deprecated):  
mpirun -n <num_ranks> python ./app.py  
  
torch.distributed.launch --nproc_per_node=4 train.py
```

```
New (recommended):  
torchrun \  
  --nnodes=4 \  
  --nproc_per_node=8 \  
  --rdzv_backend=c10d \  
  --rdzv_endpoint=node0:29500 \  
train.py
```

PyTorch – torch.distributed Hello World

- Initialization: ranks need to know their rank number
- `dist.init_process_group(backend)` sets up the process group
- With `torchrun`, environment variables are auto-set
- **Backends:**
 - `'nccl'` — GPU training (recommended)
 - `'gloo'` — CPU training or debugging
 - `'mpi'` — requires PyTorch built from source with MPI

```
import os
import torch
import torch.distributed as dist

# With torchrun, env vars are auto-set
dist.init_process_group(backend='nccl')

rank = dist.get_rank()
world_size = dist.get_world_size()
local_rank = int(os.environ['LOCAL_RANK'])

device = torch.device(f'cuda:{local_rank}')

print(f'Hello from rank {rank} '
      f'of {world_size}')

# Synchronize all ranks
dist.barrier()

dist.destroy_process_group()
```

PyTorch Supported Communications

- **Point-to-point communication:**
 - `dist.send()` and `dist.recv()` — blocking
 - `dist.isend()` and `dist.irecv()` — non-blocking (overlap with compute)
- **Collective communication:**
 - `dist.broadcast`, `dist.reduce`, `dist.all_reduce`, `dist.scatter`, `dist.gather`, `dist.all_gather`
- **Best practices:**
 - Use NCCL backend for GPU training — automatically uses NVLink/InfiniBand
 - Use Gloo for CPU training or debugging distributed logic

PyTorch – torch.distributed Point-to-Point

- **Blocking P2P:**
 - `dist.send()` / `dist.recv()`
 - Serialization of tensors done automatically
- **Non-blocking P2P:**
 - `dist.isend()` / `dist.irecv()`
 - Do NOT modify sent tensor before `req.wait()`
 - Do NOT read received tensor before `req.wait()`
 - Allows overlap of computation and communication

```
""" Blocking point-to-point """
def run(rank, size):
    tensor = torch.zeros(1)
    if rank == 0:
        tensor += 1
        dist.send(tensor=tensor, dst=1)
    else:
        dist.recv(tensor=tensor, src=0)
    print(f'Rank {rank} has {tensor[0]}')

""" Non-blocking P2P """
def run_nonblocking(rank, size):
    tensor = torch.zeros(1)
    req = None
    if rank == 0:
        tensor += 1
        req = dist.isend(tensor, dst=1)
    else:
        req = dist.irecv(tensor, src=0)
    req.wait()
    print(f'Rank {rank} has {tensor[0]}')
```

PyTorch – torch.distributed All-Reduce

- `dist.all_reduce()` — reduce and distribute to all ranks
- **Available reduce operations:**
 - `dist.ReduceOp.SUM` — used for gradient averaging
 - `dist.ReduceOp.PRODUCT`, `MAX`, `MIN`
- **Gradient averaging pattern:**
 - All-reduce the gradient sum, divide by `world_size`
 - This is exactly what PyTorch DDP does internally

```
""" Gradient averaging with AllReduce """
def average_gradients(model):
    size = float(dist.get_world_size())
    for param in model.parameters():
        dist.all_reduce(
            param.grad.data,
            op=dist.ReduceOp.SUM)
        param.grad.data /= size

""" AllReduce example """
group = dist.new_group([0, 1])
tensor = torch.ones(1).cuda()
dist.all_reduce(tensor,
    op=dist.ReduceOp.SUM,
    group=group)
print(f'Rank {dist.get_rank()}: {tensor[0]}')
```

PyTorch – torch.distributed Available Collectives

- **dist.broadcast(tensor, src, group):**
 - Copies tensor from src to all other processes
- **dist.reduce(tensor, dst, op, group):**
 - Applies op to all tensors and stores the result in dst
- **dist.all_reduce(tensor, op, group):**
 - Same as reduce, but the result is stored in all processes
- **dist.scatter(tensor, src, scatter_list, group):**
 - Copies the ith tensor scatter_list[i] to the ith process
- **dist.gather(tensor, dst, gather_list, group):**
 - Copies tensor from all processes into gather_list at dst
- **dist.all_gather(tensor_list, tensor, group):**
 - Copies tensor from all processes to tensor_list on all processes

Distributed SGD with torch.distributed

- `run()` implements forward-backward training
- `average_gradients()` uses AllReduce at each step
- This pattern is what DDP does automatically:
 - DDP hooks into backward to trigger AllReduce
 - Gradients averaged before `optimizer.step()`

```
def average_gradients(model):
    size = float(dist.get_world_size())
    for param in model.parameters():
        dist.all_reduce(param.grad.data,
                        op=dist.ReduceOp.SUM)
        param.grad.data /= size

def run(rank, size):
    torch.manual_seed(1234)
    train_set, bsz = partition_dataset()
    model = Net()
    optimizer = optim.SGD(
        model.parameters(),
        lr=0.01, momentum=0.5)
    for epoch in range(10):
        for data, target in train_set:
            optimizer.zero_grad()
            output = model(data)
            loss = F.nll_loss(output, target)
            loss.backward()
            average_gradients(model)
            optimizer.step()
```

Framework Landscape — 2026 Update

Framework	Primary Use	Key Feature	Status
PyTorch + FSDP2	Training (dominant)	DDP, FSDP2, TP, TorchTitan	✓ Dominant
DeepSpeed	Large-scale training	ZeRO 1/2/3, CPU/NVMe offload	✓ Widely used
Megatron-LM	LLM pre-training	3D parallelism (DP+TP+PP)	✓ Production
JAX / XLA	Research & Google infra	pmap/shard_map, TPU-native	✓ Growing
vLLM / SGLang	Inference serving	Paged attn, continuous batching	✓ Leading (infer.)
Caffe2 / CNTK	Historical	—	✗ Discontinued

Summary and Key Takeaways

Parameter Servers in Distributed Deep Learning

- Parameter servers were used in early distributed DL for managing and synchronizing model parameters
- **Centralized parameter servers have largely been replaced by decentralized approaches**
- **Limitations of Parameter Servers:**
 - High Communication Overhead: PS creates bottlenecks as models and workers increase
 - Scalability: PS struggles with large models, becoming inefficient with multiple servers
 - Network Bandwidth: frequent updates require high bandwidth, causing bottlenecks
- **Modern Alternatives:**
 - AllReduce (Ring) — used in PyTorch DDP, Horovod: bandwidth-optimal, no bottleneck
 - FSDP2 / ZeRO — shard params and gradients, eliminate centralized server entirely
 - Asynchronous techniques and gradient accumulation for stability

Lesson Key Points

- **Distributed DL Algorithms:**
 - Synchronous/Asynchronous — tradeoffs in accuracy and throughput
 - Centralized (Parameter Server) vs Decentralized (AllReduce)
 - Passing gradients vs parameters — ZeRO/FSDP2 for memory efficiency
- **Communication:**
 - MPI now at v4.1/5.0 — persistent collectives, partitioned communication
 - SHARP: in-network AllReduce on InfiniBand switches
- **Modern LLM-Scale Parallelism (new paradigms):**
 - 4D parallelism: DP + TP + PP + SP — required for 100B+ parameter models
 - Tensor Parallelism + DeviceMesh native in PyTorch 2.3+
- **PyTorch Distributed DL:**
 - torchrun (replaces mpirun) — fault-tolerant, elastic, SLURM-native
 - DDP → FSDP2 → TP for progressively larger models
 - vLLM / paged attention for efficient LLM inference serving

References

- [1] Matsuzaki, T. (2023). Implement Model Parallelism in LLMs. tsmatz.wordpress.com/2023/09/21/model-parallelism/
- [2] Rajbhandari, S. et al. (2020). ZeRO & DeepSpeed. Microsoft Research Blog. microsoft.com/en-us/research/blog/zero-deepspeed-new-system-optimizations-enable-training-models-with-over-100-billion-parameters/
- [3] Huang, Y. et al. (2019). GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism. NeurIPS.
- [4] Narayanan, D. et al. (2021). Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM. SC'21.
- [5] Shoeybi, M. et al. (2020). Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism.
- [6] Shazeer, N. et al. (2017). Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. ICLR.
- [7] Hugging Face. Model Parallelism Guide. huggingface.co/docs/transformers/parallelism
- [8] Awan, A. et al. (2017). S-Caffe: Co-designing MPI Runtimes and Caffe for Scalable Deep Learning. PPOPP'17.
- [9] Qi, P. et al. (2024). Zero Bubble Pipeline Parallelism. ICLR 2024.
- [10] PyTorch Distributed Overview. pytorch.org/tutorials/beginner/dist_overview.html

Attendance QR Code for 3/30

